



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

Software Requirements Specification

for BookNest – Online Bookstore Ordering and Inventory Management System

Field	Entry
Document	Jackfruit Phase-1 – Software Requirements Specification (SRS)
Project Title	BookNest – Online Bookstore Ordering and Inventory Management System
Group Number	Team 6
Course	UE24CS341A – Software Engineering (Semester V)
Technology Stack	MERN (MongoDB, Express.js, React.js, Node.js)
Customer / Stakeholder	Pageturner Books (Campus Bookstore Manager)
Version	1.1 (supersedes 1.0)
Prepared by	Team 6 – PES University, Bangalore
Date created	1 October 2026

Team

Member	SRN	Role in this document set
Dhanush Gowda A S	PES1UG24CS812	M1 – Authentication & profile module; API review
Sahajpreet Singh	PES1UG24CS397	M2 – Catalog, search & filtering; UI/UX
Rishi Jain	PES1UG24CS375	M3 – Cart, checkout, order & payment; MongoDB schema
S R Lokesh Adithya	PES1UG24CS392	M4 – Admin inventory & reporting; Testing / CI-CD

Contents

Contents	2
Revision History	3
1. Introduction	4
1.1 Purpose	4
1.2 Intended Audience and Reading Suggestions	4
1.3 Product Scope	4
1.4 References	4
2. Overall Description	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Classes and Characteristics	5
2.4 Operating Environment	5
2.5 Design and Implementation Constraints	5
2.6 Assumptions and Dependencies	5
3. External Interface Requirements	5
3.1 User Interfaces	5
3.2 Software Interfaces	5
3.3 Communications Interfaces	6
4. Analysis Models	7
4.1 Use Case Diagram	7
4.2 Use Case Descriptions	7
4.3 Entity-Relationship (Domain) Model	8
5. System Features (Functional Requirements)	9
5.1 User Account Management	9
5.2 Catalog Browsing, Search and Book Details	9
5.3 Shopping Cart	9
5.4 Order Placement and Payment	10
5.5 Order Tracking and History	10
5.6 Catalog and Inventory Management (Administrator)	11
5.7 Order Processing (Administrator)	11
5.8 Operational Reporting (Administrator)	11
6. Other Nonfunctional Requirements	12
6.1 Performance Requirements	12
6.2 Safety Requirements	12
6.3 Security Requirements	12
6.4 Software Quality Attributes	13
6.5 Business Rules and Domain Requirements	13

7. Other Requirements	13
Appendix A: Glossary	13
Appendix B: Field Layouts	14
Appendix C: Requirement Traceability Matrix	16
Appendix D: Validation Specification – Initial Planning	19
Appendix E: Project Plan (Work Breakdown)	19
Appendix F: Tool Stack	19
Appendix G: Changes from SRS v1.0 to v1.1	20

Revision History

Name	Date	Reason for changes	Version
Team 6	(v1.0 submission)	Initial SRS (Deliverable 2): functional, non-functional and security requirements, use cases, ER model, validation planning, project plan.	1.0
Team 6	1 October 2026	Aligned with the department SRS template and the Part-1 checklist: added Safety, Security Objectives, Quality Attributes, Other Requirements, per-feature stimulus/response; fixed the Order/Payment lifecycle conflict; added FR-014 to FR-016, SAF-001 to SAF-003, SEC-007, BR-01 to BR-06 identifiers; quantified NFR-001/002; completed the traceability matrix. Details in Appendix G.	1.1

1. Introduction

1.1 Purpose

This document specifies the software requirements for **BookNest**, an online bookstore ordering and inventory management system, release 1.0. It covers the whole product: the customer-facing storefront and the administrative back-office of Pageturner Books. It defines functional, non-functional, safety, security and business-rule requirements, and is the baseline for the Software Architecture & Design Specification (SAD) and the Software Test Plan (STP) that accompany it.

1.2 Intended Audience and Reading Suggestions

Developers (the four team members) should read Sections 3–7 and Appendices B–C before design. **Testers** should read Section 5, 6 and Appendix C. The **course instructor/evaluator** and the **stakeholder (Pageturner Books)** can focus on Sections 1, 2, 4 and 5. Section 2 describes the product context, Section 3 the interfaces, Section 4 the use cases and data model, Section 5 the functional requirements grouped by feature, Section 6 the quality, safety, security and business rules, Section 7 other requirements, and the appendices the glossary, field layouts, traceability, validation planning and project plan.

1.3 Product Scope

BookNest lets a customer browse and search a catalog of books, view book details, manage a shopping cart, place an order with online payment, and track order status and history. Store staff (administrator) manage the catalog and stock, move orders through their lifecycle and view sales and low-stock reports. It replaces Pageturner Books' current phone-and-counter ordering with a self-service web application, giving customers 24-hour ordering and order visibility, and giving the store an accurate digital record of sales and stock.

Out of scope for release 1.0: ratings and reviews, order cancellation and returns, e-mail/SMS notifications, multiple currencies, delivery-partner integration and a live feed from any existing inventory system.

1.4 References

- PES University – Software Engineering Project Guidelines 2026 (Jackfruit Mini-Project) and Mini-Project Deliverables Part-1.
- PES University – SRS template for SE; SE Level 3 Student Submission Guide (CoffeeKiosk exemplar).
- BookNest Synopsis (Deliverable 1), Team 6.
- BookNest Software Architecture & Design Specification (SAD) v1.0 and Software Test Plan (STP) v1.0 (companion documents in this submission).
- IEEE Std 830-1998 / ISO/IEC/IEEE 29148 (requirements practice).

2. Overall Description

2.1 Product Perspective

BookNest is a new, self-contained web application (not a follow-on to an existing system). It is a three-tier web application: a React.js single-page front end, an Express.js/Node.js REST API, and a MongoDB data store, integrated with an external (sandbox/mock) payment gateway. Figure 4.1 shows the actors and Figure 4.2 the data model; the component structure is in SAD Section 3.3.

2.2 Product Functions

- Customer account registration, login and profile management (5.1).
- Catalog browsing, search, filtering and book details (5.2).
- Shopping cart management (5.3).
- Order placement with payment (5.4).
- Order tracking and order history (5.5).
- Administrator catalog and inventory management (5.6).

- Administrator order processing – status updates (5.7).
- Administrator operational reporting (5.8).

2.3 User Classes and Characteristics

User class	Privileges	Characteristics
Visitor (customer not logged in)	Browse, search, view book details	Non-technical, anonymous; may become a Customer by registering
Customer	Visitor functions plus profile, cart, order, pay, track own orders	Non-technical; occasional to frequent use; desktop or mobile browser
Administrator	Manage catalog, stock and orders; view reports	Store staff of Pageturner Books; higher privilege; trained user (most important to satisfy for data accuracy)
Payment Gateway	Authorise payment and return success/failure	External sandbox/mock system, not a human user

2.4 Operating Environment

Server: Node.js 20 LTS with Express.js, deployed as a Docker container on a cloud host (e.g. Render or Railway). **Database:** MongoDB (Atlas free-tier replica set or self-hosted replica set). **Client:** latest versions of Chrome, Firefox, Edge and Safari on desktop or mobile, accessing the React front end over HTTPS.

2.5 Design and Implementation Constraints

- The MERN stack (MongoDB, Express.js, React.js, Node.js) is mandatory.
- Agile methodology only, with visible sprint/backlog activity in GitHub Projects or Jira.
- Git/GitHub for version control, with CI/CD, code review, static analysis and security validation evidence.
- Payment integration uses a sandbox or mock provider (Razorpay/Stripe test mode); no real card data is processed.
- All client–server traffic over HTTPS; JSON is the only API message format.

2.6 Assumptions and Dependencies

- A free-tier MongoDB Atlas cluster and a sandbox payment gateway account are available for the project duration.
- Pageturner Books' catalog data (title, author, genre, description, price, stock) is seeded/mock by the team, not taken from a live feed.
- Third-party npm packages are used for authentication (JWT, bcrypt), validation and the payment SDK.
- Currency is Indian Rupees (INR); the operating window for availability measurement is 08:00–22:00 IST (to be confirmed with the stakeholder).

3. External Interface Requirements

3.1 User Interfaces

A responsive React.js web UI with a customer storefront (catalog, book detail, cart, checkout, order tracking, order history, profile, login/register) and an admin console (catalog and stock management, order status updates, reports). Common elements on every screen: persistent navigation bar, cart icon with item count, inline form-validation messages, consistent error banners, and loading/empty/error states on every data-fetching screen. Screens are specified in SAD Section 4.5.

3.2 Software Interfaces

Component	Interface
Front end ↔ Back end	REST API over HTTPS, JSON payloads (SAD Section 4.3)
Back end ↔ Database	MongoDB through the Mongoose ODM

Component	Interface
Back end ↔ Payment Gateway	REST/SDK call to the sandbox gateway through the PaymentGateway adapter interface; the browser obtains a payment token directly from the gateway and sends only that token to BookNest

3.3 Communications Interfaces

All client–server communication uses HTTPS (TLS 1.2 or higher). Authenticated requests carry a JWT bearer token in the Authorization header. JSON is the sole message format. No e-mail or file-transfer protocol is used in release 1.0. There are no hardware interfaces (Section 3.4 of the template is not applicable).

4. Analysis Models

4.1 Use Case Diagram

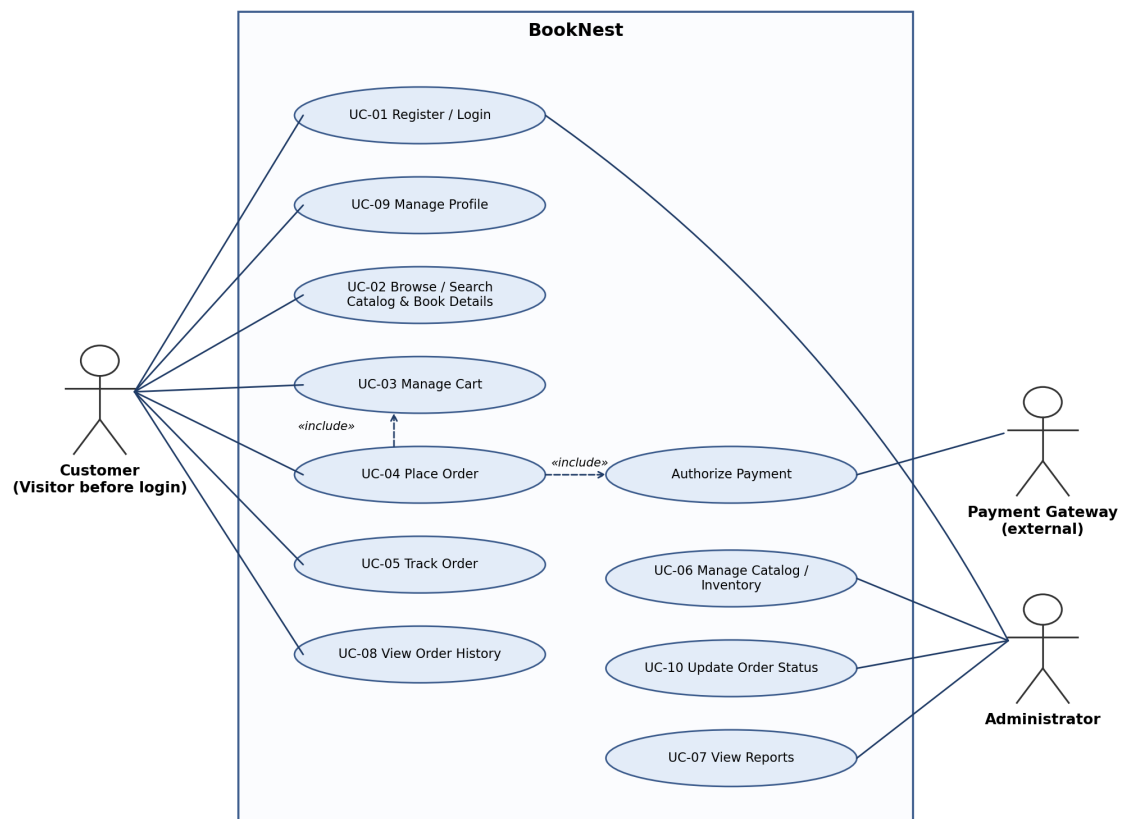


Figure 4.1 - Use case diagram (UC-02 also covers FR-015 book details; 'Visitor' = Customer who has not yet logged in)

Figure 4.1 – Use case diagram: Customer, Administrator and Payment Gateway

4.2 Use Case Descriptions

ID	Use case	Primary actor	Precondition	Main flow (summary)	Outcome	FRs
UC-01	Register / Login	Customer, Administrator	Application available	Enter details; system validates; creates account or checks credentials; issues token	Authenticated session	FR-001, FR-002
UC-02	Browse / Search Catalog, view book details	Visitor, Customer	None	Enter search text/filters; system lists matches; user opens a book	Matching books and details shown	FR-003, FR-004, FR-015
UC-03	Manage Cart	Customer	Logged in	Add book, change quantity or remove; system recalculates total	Cart updated	FR-005, FR-006
UC-04	Place Order	Customer	Logged in; valid cart	Review cart; pay; system authorises payment, then creates order and reduces stock	Order created after payment; order number shown	FR-007, FR-008, FR-012
UC-05	Track Order	Customer	Order exists	Open order; system shows current status	Current status displayed	FR-009
UC-06	Manage Catalog / Inventory	Administrator	Admin authenticated	Add, edit, disable/enable a book or adjust stock	Catalog/stock updated	FR-011
UC-07	View Reports	Administrator	Admin authenticated	Choose report and period; system calculates	Sales / low-stock report displayed	FR-013
UC-08	View Order History	Customer	Logged in	Open My Orders	Own past orders listed	FR-010
UC-09	Manage Profile	Customer	Logged in	Edit name or change password	Profile updated	FR-016
UC-10	Update Order Status	Administrator	Admin authenticated; order exists	Select order; move to next status	Customer sees new status	FR-014

4.3 Entity-Relationship (Domain) Model

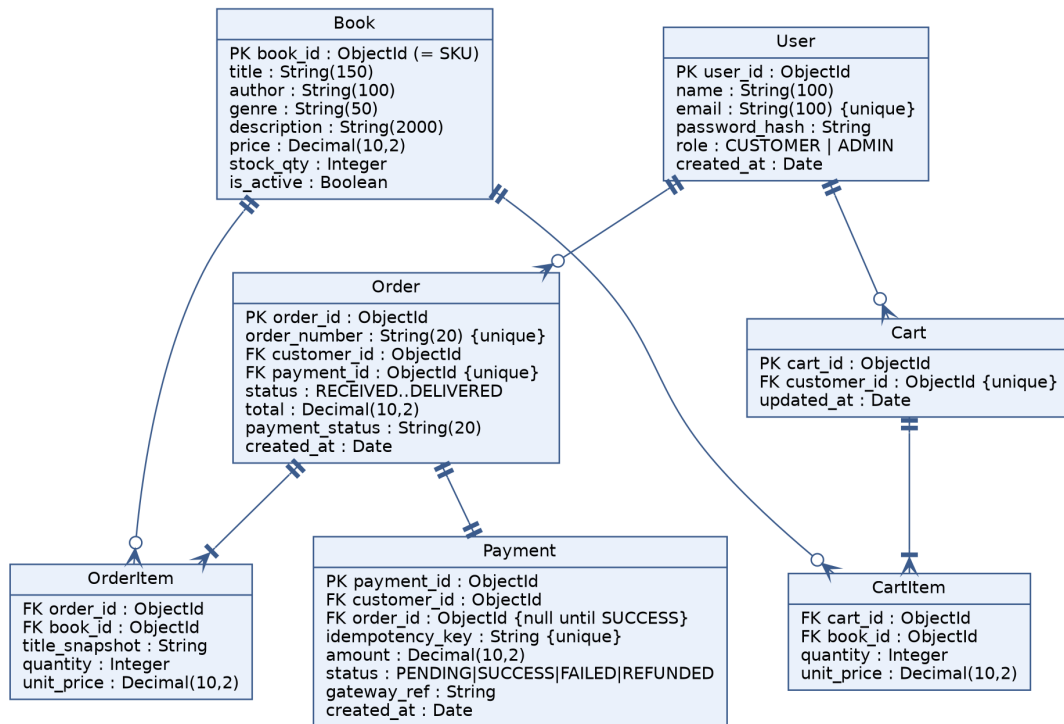


Figure 4.2 – ER model: User, Book, Cart, CartItem, Order, OrderItem, Payment

Notes: (1) CartItem and OrderItem are stored as embedded sub-documents (SAD Section 3.6). (2) A Payment is created first with status PENDING and a unique idempotency key; *Payment.order_id* stays empty until the payment succeeds. An Order exists only after a successful payment. (3) The class diagram, sequence, activity, state, component and deployment views are in the SAD (Sections 3.3, 4.1 and 4.2).

5. System Features (Functional Requirements)

Each requirement is uniquely tagged and is written to be concise, complete, unambiguous, verifiable and necessary. Priority is High, Medium or Low. Verification names the planned method (details in the STP).

5.1 User Account Management

5.1.1 Description and Priority

Allows visitors to create an account, log in and keep their profile up to date. Authentication is required for cart, order and administrator functions. **Priority: High.**

5.1.2 Stimulus/Response Sequences

- Visitor submits registration form → system validates fields, stores the account (password hashed) and confirms.
- Registered customer submits email and password → system returns a session token and opens the storefront.
- Customer edits name or password → system validates and saves.

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a visitor to create a customer account using a unique email address and a password of at least 8 characters.	High	Functional test
FR-002	The system shall allow a registered customer to log in with email and password and receive a signed session token (JWT).	High	Functional test
FR-016	The system shall allow a logged-in customer to view and update their name and password; email and role cannot be changed through this function.	Low	Functional test

Error conditions: duplicate email (409); invalid or missing fields (400 with per-field messages); wrong credentials (401, same message for unknown user and wrong password); too many failed logins (429, SEC-007).

5.2 Catalog Browsing, Search and Book Details

5.2.1 Description and Priority

Lets any visitor find books by title, author or genre, narrow results by price and availability, and read the details of a book. **Priority: High.**

5.2.2 Stimulus/Response Sequences

- Visitor enters search text or selects a genre → system lists matching active books, 12 per page.
- Visitor sets price range or 'available only' → system narrows the list.
- Visitor opens a book → system shows title, author, genre, description, price and availability.

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-003	The system shall allow any visitor to browse and search the book catalog by title, author or genre.	High	Functional test
FR-004	The system shall allow a visitor to filter catalog results by price range (inclusive bounds) and availability.	Medium	Functional test
FR-015	The system shall allow any visitor to view a book's detail page showing title, author, genre, description, price and availability.	Medium	Functional test

Error conditions: no match (empty list with a 'no results' message); unknown or disabled book id (404); operator characters in search text are treated as plain text.

5.3 Shopping Cart

5.3.1 Description and Priority

Lets a customer collect books before paying. The cart total is always calculated by the server. **Priority: High.**

5.3.2 Stimulus/Response Sequences

- Customer adds a book → system adds it (or increases quantity) and updates the total.
- Customer changes quantity or removes a line → system updates the cart and total.

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-005	The system shall allow a customer to add an available book to their cart, update its quantity (up to the available stock), and remove it.	High	Functional test
FR-006	The system shall calculate the cart total from item price and quantity, exactly to two decimal places.	High	Unit + system test

Error conditions: book unavailable or disabled (409); quantity zero, negative, non-integer or above stock (400/409); cart of another customer is never accessible.

5.4 Order Placement and Payment

5.4.1 Description and Priority

Turns a cart into a confirmed order. Payment is authorised first; the order is created and stock reduced only if payment succeeds. **Priority: High.**

5.4.2 Stimulus/Response Sequences

- Customer presses Pay → system re-checks the cart and stock, recomputes the total, creates a PENDING payment and asks the gateway to authorise it.
- Gateway approves → system creates the order (RECEIVED) with an order number, reduces stock, clears the cart and shows the confirmation.
- Gateway declines or times out → system records the failure, creates no order, keeps the cart and allows a retry.

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-007	The system shall create an order only after payment has been authorised successfully.	High	Negative + positive test
FR-008	The system shall assign a unique order number to every successfully created order.	High	Integration test
FR-012	The system shall record payment success or failure (and refund where applicable) against the corresponding payment record and, on success, the order.	High	Integration test

Error conditions: empty cart (400); an item became unavailable (409, no payment taken); payment declined (402); gateway timeout followed by retry with the same idempotency key (original result returned, no second order or charge); last copy bought by two customers at once (one order, the other payment refunded).

5.5 Order Tracking and History

5.5.1 Description and Priority

Lets a customer see where an order stands and review earlier purchases. **Priority: High.**

5.5.2 Stimulus/Response Sequences

- Customer opens an order → system shows items, total and current status (RECEIVED, PROCESSING, SHIPPED, DELIVERED).
- Customer opens My Orders → system lists own orders, newest first, with paging.

5.5.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-009	The system shall allow a customer to view the current status of their own order (RECEIVED, PROCESSING, SHIPPED or DELIVERED).	High	System test
FR-010	The system shall allow a customer to view their past order history, newest first.	Medium	Functional test

Error conditions: order of another customer or unknown id (404); malformed id (400); no login (401).

5.6 Catalog and Inventory Management (Administrator)

5.6.1 Description and Priority

Lets the administrator keep the catalog and stock accurate. **Priority: High.**

5.6.2 Stimulus/Response Sequences

- Administrator adds or edits a book → system validates and saves; it appears in the catalog.
- Administrator disables or enables a listing → system hides/shows it to customers.
- Administrator adjusts stock → system updates quantity.

5.6.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-011	The administrator shall be able to add, edit and disable (and re-enable) book listings, including adjusting stock quantity; stock shall never become negative.	High	Functional test

Error conditions: invalid price, stock or missing title (400); stock adjustment below zero (400/409); non-administrator (403).

5.7 Order Processing (Administrator)

5.7.1 Description and Priority

Lets the administrator move paid orders through fulfilment so customers can follow progress. **Priority: High.**

5.7.2 Stimulus/Response Sequences

- Administrator selects an order and the next status → system validates the step and saves it.
- Customer reloads the order → system shows the new status.

5.7.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-014	The administrator shall be able to move an order forward through RECEIVED, PROCESSING, SHIPPED and DELIVERED, one step at a time and in that order only; the customer shall see the new status.	High	Functional + negative test

Error conditions: skipped or backward step (409); unknown status value (400); unknown order (404); non-administrator (403).

5.8 Operational Reporting (Administrator)

5.8.1 Description and Priority

Gives the administrator basic sales and stock information. **Priority: Medium.**

5.8.2 Stimulus/Response Sequences

- Administrator selects a period → system shows number of orders and revenue.
- Administrator opens the low-stock report → system lists books at or below the threshold (default 5), lowest first.

5.8.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-013	The administrator shall be able to view a sales summary (order count and revenue for a chosen period, counting only successful payments) and a low-stock report.	Medium	Functional test

Error conditions: invalid or reversed dates (400); no orders in period (zero totals); non-administrator (403).

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Measurable requirement	Rationale	Verification
NFR-001	95% of catalog search requests shall receive a response within 2 seconds with 50 concurrent users and a catalog of 500 books; error rate below 1%.	Browsing speed decides whether customers continue to checkout; the figures match a single store on free-tier hosting.	Performance test (Artillery)

6.2 Safety Requirements

BookNest cannot injure people, but it can cause financial harm (wrong charges) and data loss. These requirements prevent such harm.

ID	Requirement	Verification
SAF-001	A customer shall never be charged for an order that does not exist: if payment is authorised but the order cannot be created, the payment shall be refunded automatically in the same request, and a retry of the same checkout shall never create a second charge or order.	Integration/negative test
SAF-002	Book stock quantity shall never be negative, even when two customers buy the last copy at the same time.	Concurrency test
SAF-003	Application data shall be backed up daily (logical export retained 7 days) and a backup shall be restorable to an empty database (maximum data loss 24 hours).	Operational test

6.3 Security Requirements

Security objectives

ID	Objective	Addressed by
SO-1	Confidentiality – only the owner can see their account, cart, orders and personal data; passwords and payment data are never exposed.	SEC-001, SEC-005
SO-2	Integrity – prices, totals, stock and order status can be changed only through validated, authorised operations; user input cannot alter queries or pages.	SEC-002, SEC-003
SO-3	Authentication and authorisation – only registered users act as customers; only administrators perform administrative functions.	SEC-002, SEC-004, SEC-007
SO-4	Accountability and availability – security-relevant events are recorded and brute-force abuse is limited.	SEC-006, SEC-007

Security requirements

ID	Security requirement	Validation approach
SEC-001	Passwords shall be hashed with bcrypt (cost 10 or higher) and never stored or logged in plaintext.	Code/configuration review; unit test
SEC-002	Role-based access control shall prevent customers and anonymous users from accessing administrator endpoints (403 or 401).	Authorization tests; ZAP scan
SEC-003	The system shall validate and sanitise all user-supplied fields (body, query, path) before processing and shall escape them on output.	Negative/security tests; SonarQube
SEC-004	Authenticated APIs shall reject expired, tampered or missing JWTs (1-hour expiry), and all traffic shall use HTTPS with TLS 1.2 or higher.	Security tests; TLS check
SEC-005	Sensitive payment information (card numbers, CVV) shall not be stored or logged by the application; only status and gateway reference are retained.	Design review; DB/log inspection
SEC-006	Security-relevant events (failed login, unauthorised access, invalid token) shall be logged with timestamp, user (if known), IP and route, without passwords or tokens.	System test
SEC-007	The system shall limit failed logins to 5 per account and IP in 15 minutes; further attempts are refused (HTTP 429).	Security test

6.4 Software Quality Attributes

ID	Attribute	Measurable requirement	Verification
NFR-002	Availability	The application shall be available at least 99% of the time during the operating window 08:00–22:00 IST daily, excluding announced maintenance.	Operational review (uptime monitor)
NFR-003	Usability	A new customer shall be able to place a basic order in no more than 5 primary screens (Catalog, Book Detail, Cart, Checkout, Confirmation).	Usability walkthrough
NFR-004	Reliability	An order shall not be created twice when a customer retries after a payment timeout.	Negative/integration test
NFR-005	Maintainability	Business logic shall be separated into identifiable Express service modules with automated unit tests (line coverage of services at least 80%).	Code review; coverage report
NFR-006	Testability	Core services shall be testable without the live payment gateway through a mockable adapter.	Unit/integration test

Relative preferences: correctness of money and stock outweighs raw speed; security outweighs convenience (for example, no 'remember me' or long-lived tokens in release 1.0); ease of use for first-time customers outweighs feature richness.

6.5 Business Rules and Domain Requirements

ID	Rule
BR-01	Only books marked available (active and in stock) can be added to a cart or ordered.
BR-02	A customer cannot modify an order once it has moved past RECEIVED status.
BR-03	Only an administrator can disable a book listing or adjust stock manually.
BR-04	An order must contain at least one valid item.
BR-05	A payment failure shall not result in a confirmed order or a stock deduction.
BR-06	Stock quantity is decremented only after payment is confirmed successful.
DR-01	Order status moves only forward: RECEIVED, PROCESSING, SHIPPED, DELIVERED.
DR-02	Prices are in INR with two decimal places; an order keeps the prices that applied when it was placed.
DR-03	A customer has one active cart.

7. Other Requirements

Area	Requirement
Database	MongoDB with collections users, books, carts, orders, payments, securitylogs (SAD 3.6). Orders and payments are retained for at least 3 years (assumption to be confirmed); securitylogs for 90 days; backups per SAF-003.
Legal / privacy	Only name, email and password hash are collected as personal data; no card data is stored. Handling follows the principles of the Indian Digital Personal Data Protection Act, 2023 (purpose limitation, minimal data). A privacy notice is shown at registration.
Localization	English user interface; INR currency; times shown in IST and stored in UTC.
Deployment	Delivered as a Docker image with configuration from environment variables; static front end served over HTTPS.
Reuse	Open-source npm libraries for JWT, bcrypt, validation and rate limiting; the PaymentGateway adapter allows the provider to be replaced.

Appendix A: Glossary

Term	Definition
Catalog	The full set of book listings available for browsing/search.
Cart	A customer's temporary, pre-checkout collection of selected books.

Term	Definition
Order	A confirmed, paid request to purchase one or more books.
SKU	Stock keeping unit – unique identifier of a book listing (the book_id).
RBAC	Role-Based Access Control – restricting actions by role (customer, administrator).
JWT	JSON Web Token – signed token used to authenticate API requests.
Idempotency key	Unique value sent with a checkout request so that repeating the request cannot create a second order or charge.
Visitor	A customer who has not logged in.
Payment Gateway	External service that authorises payments (sandbox/mock in this project).
ODM	Object-Document Mapper (Mongoose).

Appendix B: Field Layouts

Field layouts (length, data type, mandatory flag). Lengths are maximum characters; Decimal(10,2) means 10 digits with 2 after the point.

B.1 Book

Field	Length	Data type	Description	Mandatory
book_id	24	ObjectId	Unique book identifier (SKU)	Y
title	150	String	Book title	Y
author	100	String	Author name	Y
genre	50	String	Genre/category	N
description	2000	String	Short description	N
price	10,2	Decimal	Selling price (INR)	Y
stock_qty	6	Integer	Current stock (≥ 0)	Y
is_active	1	Boolean	Whether listing is enabled	Y

B.2 User

Field	Length	Data type	Description	Mandatory
user_id	24	ObjectId	Unique user identifier	Y
name	100	String	Full name	Y
email	100	String	Unique login email	Y
password_hash	60	String	bcrypt hash	Y
role	10	Enum	CUSTOMER / ADMIN	Y
created_at	8	Date	Registration date/time	Y

B.3 Order and Payment

Field	Length	Data type	Description	Mandatory
order_id	24	ObjectId	Unique order identifier	Y
order_number	20	String	Human-readable unique number (BN-YYYYMMDD-nnnn)	Y
customer_id	24	ObjectId	Reference to customer	Y
status	20	Enum	RECEIVED / PROCESSING / SHIPPED / DELIVERED	Y
total	10,2	Decimal	Order total (INR)	Y
payment_id	24	ObjectId	Reference to successful payment	Y
payment_status	20	Enum	SUCCESS at creation	Y

Field	Length	Data type	Description	Mandatory
created_at	8	Date	Order date/time	Y
payments.idempotency_key	64	String	Unique per checkout attempt	Y
payments.status	20	Enum	PENDING / SUCCESS / FAILED / REFUNDED	Y
payments.gateway_ref	64	String	Reference returned by gateway	N
payments.amount	10,2	Decimal	Amount authorised	Y

B.4 Report requirements

Sales Summary Report	Low-Stock Report	Order History (customer view)
Period (from, to) Number of orders Total revenue (INR) Average order value (successful payments only)	Book title Author Genre Current stock Threshold used	Order number Order date Number of items Total Current status

Appendix C: Requirement Traceability Matrix

Requirement → architecture → design → code → test. Architecture and design references are sections of the SAD v1.0; test case IDs are in the STP v1.0 (Section 13). Code references are the planned repository paths and are replaced with actual file paths as each feature is merged.

Sl.	Req ID	Brief description	Architecture ref	Design ref	Code / file ref (planned)	Test Case IDs	System Test ID
1	FR-001	Customer registration	SAD 3.4 AuthService	SAD 4.1.2, 4.2.3, 4.3.1	server/src/services/authService.js; client/src/pages/Register.jsx	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03	ST-01
2	FR-002	Customer login / JWT	SAD 3.4 AuthService; 3.9	SAD 4.2.3, 4.3.1	server/src/services/authService.js; middleware/auth.js	TC-AUTH-04, TC-AUTH-05	ST-01
3	FR-003	Browse and search catalog	SAD 3.4 CatalogService	SAD 4.1.2, 4.3.2	server/src/services/catalogService.js; client/src/pages/Catalog.jsx	TC-CAT-01, TC-CAT-02, TC-CAT-03, TC-CAT-04, TC-CAT-09	ST-01, ST-02
4	FR-004	Filter by price and availability	SAD 3.4 CatalogService	SAD 3.6 (indexes), 4.3.2	server/src/services/catalogService.js	TC-CAT-05, TC-CAT-06	–
5	FR-005	Add, update, remove cart items	SAD 3.4 CartService	SAD 4.1.1, 4.1.2, 4.3.3	server/src/services/cartService.js; client/src/pages/Cart.jsx	TC-CART-01, TC-CART-02, TC-CART-03, TC-CART-04, TC-CART-05	ST-02
6	FR-006	Cart total calculation	SAD 3.4 CartService	SAD 4.1.1 (Cart.total)	server/src/utills/money.js; models/Cart.js	TC-CART-06, TC-CART-07, TC-ORD-09, TC-TRK-06	ST-02
7	FR-007	Order only after payment authorised	SAD 3.4 OrderService, PaymentService	SAD 4.1.3, 4.2.1, 4.3.4	server/src/services/orderService.js	TC-ORD-01, TC-ORD-02, TC-ORD-05, TC-ORD-08, TC-ORD-09	ST-02, ST-03, ST-04
8	FR-008	Unique order number	SAD 3.4 OrderService	SAD 3.6 (unique index), 4.2.1	server/src/services/orderService.js; models/Order.js	TC-ORD-01, TC-ORD-06	ST-02
9	FR-009	View order status	SAD 3.4 OrderService	SAD 4.1.4, 4.3.4	server/src/controllers/orderController.js ; client/src/pages/OrderDetail.jsx	TC-TRK-01, TC-TRK-02, TC-TRK-03, TC-TRK-04, TC-TRK-05, TC-TRK-06	ST-02, ST-06
10	FR-010	Order history	SAD 3.4 OrderService	SAD 4.3.4	server/src/controllers/orderController.js ; client/src/pages/Orders.jsx	TC-TRK-07, TC-TRK-08, TC-TRK-09	ST-08
11	FR-011	Admin add/edit/disable books, stock	SAD 3.4 InventoryService	SAD 4.1.2, 4.3.5	server/src/services/inventoryService.js; client/src/pages/admin/Books.jsx	TC-ADM-01, TC-ADM-02, TC-ADM-03, TC-ADM-04, TC-ADM-05, TC-ADM-06, TC-ADM-07	ST-05
12	FR-012	Record payment success/failure	SAD 3.4 PaymentService	SAD 4.1.3, 4.1.4, 4.2.1	server/src/services/paymentService.js; models/Payment.js	TC-ORD-01, TC-ORD-02, TC-RPT-03	ST-03
13	FR-013	Sales summary and low-stock report	SAD 3.4 ReportService	SAD 4.3.6	server/src/services/reportService.js; client/src/pages/admin/Reports.jsx	TC-RPT-01, TC-RPT-02, TC-RPT-03, TC-RPT-04, TC-RPT-05, TC-RPT-06, TC-RPT-07, TC-RPT-08	ST-08
14	FR-014	Admin updates order status	SAD 3.4 OrderService	SAD 4.1.4, 4.2.2, 4.3.5	server/src/services/orderService.js (advanceStatus)	TC-TRK-05, TC-OPS-01, TC-OPS-02, TC-OPS-03, TC-OPS-04, TC-OPS-05, TC-OPS-06, TC-OPS-08	ST-06
15	FR-015	Book detail view	SAD 3.4 CatalogService	SAD 4.3.2	server/src/services/catalogService.js; client/src/pages/BookDetail.jsx	TC-CAT-07	ST-02
16	FR-016	View/update profile and password	SAD 3.4 AuthService	SAD 4.3.1	server/src/controllers/userController.js; client/src/pages/Profile.jsx	TC-AUTH-10, TC-AUTH-11, TC-AUTH-12, TC-AUTH-13, TC-AUTH-14, TC-AUTH-15, TC-AUTH-16, TC-AUTH-17	–
17	NFR-001	Search response time (p95 <= 2 s)	SAD 3.5, 3.6	SAD 3.6 (indexes), ADR-07	models/Book.js indexes; tests/perf/search.yml (Artillery)	TC-PERF-01	–

Sl.	Req ID	Brief description	Architecture ref	Design ref	Code / file ref (planned)	Test Case IDs	System Test ID
18	NFR-002	Availability >= 99%	SAD 3.3.1, 3.5	SAD 3.3.1 (deployment), 4.4 (health)	server/src/routes/health.js; Dockerfile	TC-AVL-01	–
19	NFR-003	Order in <= 5 screens	SAD 3.2	SAD 4.5 (UX)	client/src/pages/*; client/src/App.jsx	TC-CAT-09, TC-USE-01	ST-02
20	NFR-004	No duplicate order on retry	SAD 3.5	SAD 4.1.3, 4.2.1, ADR-04	services/orderService.js; unique index Payment.idempotency_key	TC-ORD-04, TC-ORD-05	ST-04
21	NFR-005	Modular business logic + unit tests	SAD 3.4, 3.5	SAD 4.1.2, ADR-01	server/src/services/*; server/tests/unit/*	TC-MNT-01	–
22	NFR-006	Mockable payment adapter	SAD 3.4 PaymentGateway	SAD 4.1.2, ADR-03	server/src/adapters/{paymentGateway, mockGateway, razorpayGateway}.js	TC-ORD-10	–
23	SEC-001	Password hashing	SAD 3.9	SAD 3.9 (STRIDE)	server/src/services/authService.js (bcrypt)	TC-AUTH-06, TC-AUTH-13	–
24	SEC-002	RBAC on admin endpoints	SAD 3.9	SAD 4.3.5, 3.9	server/src/middleware/rbac.js; routes/admin.js	TC-CART-08, TC-TRK-02, TC-OPS-05, TC-ADM-08, TC-RPT-08, TC-SEC-01, TC-SEC-08	ST-07
25	SEC-003	Input validation and sanitising	SAD 3.9	SAD 3.9, 4.4	server/src/middleware/validate.js; client form validation	TC-AUTH-03, TC-AUTH-12, TC-AUTH-15, TC-CAT-08, TC-OPS-06, TC-ADM-02, TC-RPT-05, TC-SEC-03, TC-SEC-05, TC-SEC-07, TC-SEC-08	–
26	SEC-004	Reject expired/invalid JWT; HTTPS	SAD 3.9	SAD 3.9, 4.2.3	server/src/middleware/auth.js	TC-AUTH-07, TC-AUTH-08, TC-AUTH-14, TC-TRK-04, TC-SEC-05, TC-SEC-06	ST-07
27	SEC-005	No card data stored	SAD 3.9	SAD 4.1.5 (Payment fields), 4.2.1	server/src/models/Payment.js; adapters/*	TC-SEC-04	–
28	SEC-006	Security event logging	SAD 3.9	SAD 4.4, 3.6 (securitylogs)	server/src/middleware/securityLogger.js	TC-AUTH-05, TC-SEC-02	ST-07
29	SEC-007	Login rate limiting	SAD 3.9	SAD 3.9	server/src/middleware/rateLimit.js	TC-AUTH-09	–
30	SAF-001	No charge without an order (auto-refund)	SAD 3.4 OrderService, PaymentService	SAD 4.1.3, 4.2.1	server/src/services/orderService.js; paymentService.js	TC-ORD-04, TC-ORD-05, TC-ORD-08	–
31	SAF-002	Stock never negative	SAD 3.4 InventoryService	SAD 4.1.3, ADR-08	server/src/services/inventoryService.js (conditional update)	TC-ORD-07, TC-ORD-08, TC-ADM-05	–
32	SAF-003	Daily backup and restore	SAD 3.3.1, 3.7	SAD 3.6, 3.7	scripts/backup.sh; .github/workflows/backup.yml	TC-SAF-01	–
33	BR-01	Only available books can be carted/ordered	SAD 3.4 CartService	SAD 4.1.3	server/src/services/cartService.js	TC-CART-01, TC-CART-03, TC-ORD-11, TC-ADM-06	ST-05
34	BR-02	No order modification after RECEIVED	SAD 3.4 OrderService	SAD 4.1.4	server/src/services/orderService.js	TC-OPS-07	ST-06
35	BR-03	Only admin disables books / adjusts stock	SAD 3.9	SAD 4.3.5	server/src/routes/admin.js	TC-ADM-04, TC-ADM-06, TC-ADM-08, TC-SEC-01	ST-05
36	BR-04	Order needs at least one valid item	SAD 3.4 OrderService	SAD 4.1.3	server/src/services/orderService.js	TC-ORD-03, TC-ORD-11	–

Sl.	Req ID	Brief description	Architecture ref	Design ref	Code / file ref (planned)	Test Case IDs	System Test ID
37	BR-05	Payment failure: no order, no stock change	SAD 3.4 OrderService	SAD 4.1.3, 4.2.1	server/src/services/orderService.js	TC-ORD-02, TC-ORD-08	ST-03
38	BR-06	Stock decremented only after payment success	SAD 3.4 InventoryService	SAD 4.1.3, 4.2.1	server/src/services/inventoryService.js	TC-ORD-01, TC-ORD-07	–

Appendix D: Validation Specification – Initial Planning

At this stage validation planning shows how each requirement will be validated; it is not a test report. Detailed approach, security validation and test cases are in the STP.

Requirement category	Planned validation	Example
Functional	Black-box functional tests	Valid/invalid order placement
Security	Authentication/authorisation tests, input validation tests, static analysis, planned penetration scan (OWASP ZAP)	Customer attempts an admin endpoint
Performance	Load/response-time testing	Concurrent catalog searches
Usability	Task-based walkthrough	New user places an order in 5 screens or fewer
Reliability / safety	Failure and retry scenarios, concurrency tests, backup restore	Payment timeout and retry; last-copy purchase
Unit	Automated unit tests with line and branch coverage	Cart total calculation, stock decrement logic

Appendix E: Project Plan (Work Breakdown)

Lifecycle: Agile (Scrum) with six sprints (see Synopsis 1.6); justified by evolving requirements, frequent demos and the course requirement for sprint-based delivery. M1 = Dhanush Gowda A S, M2 = Sahajpreet Singh, M3 = Rishi Jain, M4 = S R Lokesh Adithya.

Work package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements + use cases	All	4	SRS
WP2	Architecture + UML	M1 / M2	4	SAD
WP3	Customer auth and profile module (FR-001, 002, 016)	M1	6	Working feature
WP4	Catalog/search module (FR-003, 004, 015)	M2	6	Working feature
WP5	Cart/order/payment module (FR-005–010, 012)	M3	7	Working feature
WP6	Admin inventory, order status, reporting (FR-011, 013, 014)	M4	7	Working feature
WP7	Testing + CI/CD + security	All (lead M4)	6	Test/validation evidence
WP8	Final validation + report	All	4	Final report

Appendix F: Tool Stack

Activity	Tool	Evidence to submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning / bug tracking	GitHub Projects (auto-add, PR-linked status) / Jira	Backlog, user stories, sprint board, issues
CI/CD	GitHub Actions / Jenkins	Pipeline run/log
Containerization	Docker	Dockerfile + build/run evidence
Testing	Jest, Supertest, React Testing Library, Artillery	Test and coverage reports
Static analysis	SonarQube or similar	Analysis report
API testing	Postman	Collection/results
Design	Draw.io / StarUML / PlantUML	UML diagrams

Appendix G: Changes from SRS v1.0 to v1.1

Ref	Finding in v1.0	Resolution in v1.1
G-1	Order/Payment lifecycle conflict (mandatory Payment.order_id vs order only after payment).	Payment created first with idempotency key; order_id set on success; REFUNDED status added.
G-2	Stakeholder named differently in Synopsis (Campus Bookstore Manager) and SRS (Pageturner Books); 'Payment Service' vs 'Payment Gateway'; undefined 'visitor'.	Single naming: Pageturner Books (Campus Bookstore Manager); Payment Gateway; Visitor defined in 2.3.
G-3	No admin function to change order status; no book detail, profile or cart removal requirement although promised in the Synopsis.	FR-014, FR-015, FR-016 added; FR-005 amended.
G-4	NFR-001/NFR-002 used undefined 'test load' and 'operating window'.	Quantified (50 users, 500 books; 08:00–22:00 IST).
G-5	Template sections missing: Safety, Security objectives, Quality attributes (separate), Other requirements, per-feature stimulus/response, revision history.	Added (6.2, 6.3, 6.4, 7, 5.x.2, Revision History); safety requirements SAF-001 to SAF-003 and SEC-007 added.
G-6	Business rules had no IDs; use-case table lacked order history, profile and status update; ER lacked order_number and description.	BR-01 to BR-06 and DR-01 to DR-03; UC-08 to UC-10; ER and field layouts updated.
G-7	Appendix C had 5 of ~30 rows, all TBD; M1–M4 undefined.	Complete matrix (Appendix C); owners defined in Appendix E.



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

Software Architecture and Design Specification

BookNest – Online Bookstore Ordering and Inventory Management System

Field	Entry
Project	BookNest – Online Bookstore Ordering and Inventory Management System
Version	1.0
Authors	Team 6 (Dhanush Gowda A S, Sahajpreet Singh, Rishi Jain, S R Lokesh Adithya)
Date	1 October 2026
Status	Final for Part-1 submission
Course	UE24CS341A – Software Engineering (Semester V)
Technology Stack	MERN (MongoDB, Express.js, React.js, Node.js)
Customer / Stakeholder	Pageturner Books (Campus Bookstore Manager)

Revision History

Version	Date	Author	Change Summary
1.0	1 October 2026	Team 6	First complete architecture and design specification, based on SRS v1.1

Approvals

Role	Name	Signature / Date
Architect (M1, M2)	Dhanush Gowda A S / Sahajpreet Singh	
Design lead (M3)	Rishi Jain	
Test / CI lead (M4)	S R Lokesh Adithya	
Instructor		

Contents

Contents	2
1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Audience	3
1.4 Definitions	3
2. Document Overview	3
2.1 How to use this document	3
2.2 Related Documents	3
3. Architecture	3
3.1 Goals & Constraints	3
3.2 Stakeholders & Concerns	4
3.3 Component (UML) Diagram	4
3.4 Component Descriptions	5
3.5 Chosen Architecture Pattern and Rationale	6
3.6 Technology Stack & Data Stores	7
3.7 Risks & Mitigations	7
3.8 Traceability to Requirements	8
3.9 Security Architecture	8
4. Design	10
4.1 Design Overview	10
4.2 UML Sequence Diagrams	14
4.3 API Design	17
4.4 Error Handling, Logging & Monitoring	19
4.5 UX Design	19
4.6 Open Issues & Next Steps	19
5. Appendices	20
5.1 Glossary	20
5.2 References	20
5.3 Tools	20

1. Introduction

1.1 Purpose

This document specifies the architecture and design of BookNest, the online bookstore ordering and inventory management system for Pageturner Books. It turns the requirements of the SRS (v1.1) into structures the team can implement and test: components, architecture pattern, technology, data stores, security architecture, UML views, APIs, error handling and UX.

1.2 Scope

Covers release 1.0: customer services (registration/login/profile, catalog search and details, cart, checkout with payment, order tracking and history) and administrator services (catalog and stock management, order status updates, sales and low-stock reports). Out of scope: ratings/reviews, cancellations and returns, notifications, multiple currencies.

1.3 Audience

Developers, QA engineers, security reviewers, the course instructor/evaluators and the stakeholder (Pageturner Books) who maintains the system after hand-over.

1.4 Definitions

MERN – MongoDB, Express.js, React.js, Node.js; REST – Representational State Transfer; JWT – JSON Web Token; RBAC – role-based access control; ODM – object-document mapper; ADR – architecture decision record; STRIDE – Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; Idempotency key – unique value that makes a repeated request safe; Txn – MongoDB multi-document transaction. Further terms: Appendix 5.1 and SRS Appendix A.

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture (goals, stakeholders, components, pattern, technology, risks, traceability, security). Section 4 gives the design (class, activity and state views, three sequence diagrams, API contracts, error handling, UX). Architecture decision records are in 3.5, threat modelling (STRIDE) in 3.9, and the API design in 4.3.

2.2 Related Documents

- Software Requirements Specification (SRS) v1.1 – requirement IDs FR-, NFR-, SAF-, SEC-, BR- are used throughout.
- Software Test Plan (STP) v1.0 – test cases that verify this design.
- Requirement Traceability Matrix (RTM) – SRS Appendix C (requirement → architecture → design → code → test).
- BookNest Synopsis (Deliverable 1) – ownership matrix and sprint plan.

3. Architecture

3.1 Goals & Constraints

Goal	Source	Architectural response
Correct payment and order behaviour; no double charge or negative stock	FR-007, FR-012, NFR-004, SAF-001, SAF-002, BR-05, BR-06	Payment-first checkout with idempotency key (unique index), server-side totals, conditional atomic stock decrement inside a MongoDB transaction, automatic refund if stock is lost in a race (4.1.3, 4.2.1; ADR-04, ADR-08).

Goal	Source	Architectural response
Responsive catalog search	NFR-001	Indexed queries, pagination (12 per page), projections, connection pooling, stateless API (ADR-07).
Availability >= 99%	NFR-002, SAF-003	Stateless JWT authentication, containerised API with restart policy, managed MongoDB replica set, /api/health monitored, daily backups.
Usability: order in <= 5 screens	NFR-003	Purchase path Catalog, Book Detail, Cart, Checkout, Confirmation (4.5).
Maintainability and testability	NFR-005, NFR-006	Strict layering and one service per business area; PaymentGateway interface with a mock adapter (ADR-01, ADR-03).
Security	SEC-001 to SEC-007	Middleware chain, bcrypt, RBAC, validation, rate limiting, no card data, security log (3.9).

Constraints: MERN stack is mandatory; Agile delivery with GitHub Projects/Jira; Git/GitHub with CI/CD and code review; payment through a sandbox or mock provider; catalog data seeded by the team; free-tier hosting and database (limited memory, possible cold starts, 512 MB storage); all traffic over HTTPS.

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed in
Customers	Fast search, simple ordering, safe payment, privacy of personal data, visible order status	3.1, 3.9, 4.5
Pageturner Books manager / administrator	Accurate stock and prices, simple admin screens, reports, system available during opening hours	3.4, 4.3.5, 4.3.6, 3.7
Developers (Team 6)	Modularity, clear ownership, testability, CI feedback	3.4, 3.5, 4.1
Instructor / evaluators	Traceability from requirement to test, documentation quality, explainable design, security validation	3.8, 3.9, STP
Payment provider / privacy rules	No card data in BookNest, minimal personal data	3.9, 4.2.1

3.3 Component (UML) Diagram

Figure 3.1 shows the components and their dependencies. Dependencies point downward only: a controller may call a service, a service may call a model or the PaymentGateway interface, never the reverse. The domain and service class diagrams are in 4.1.

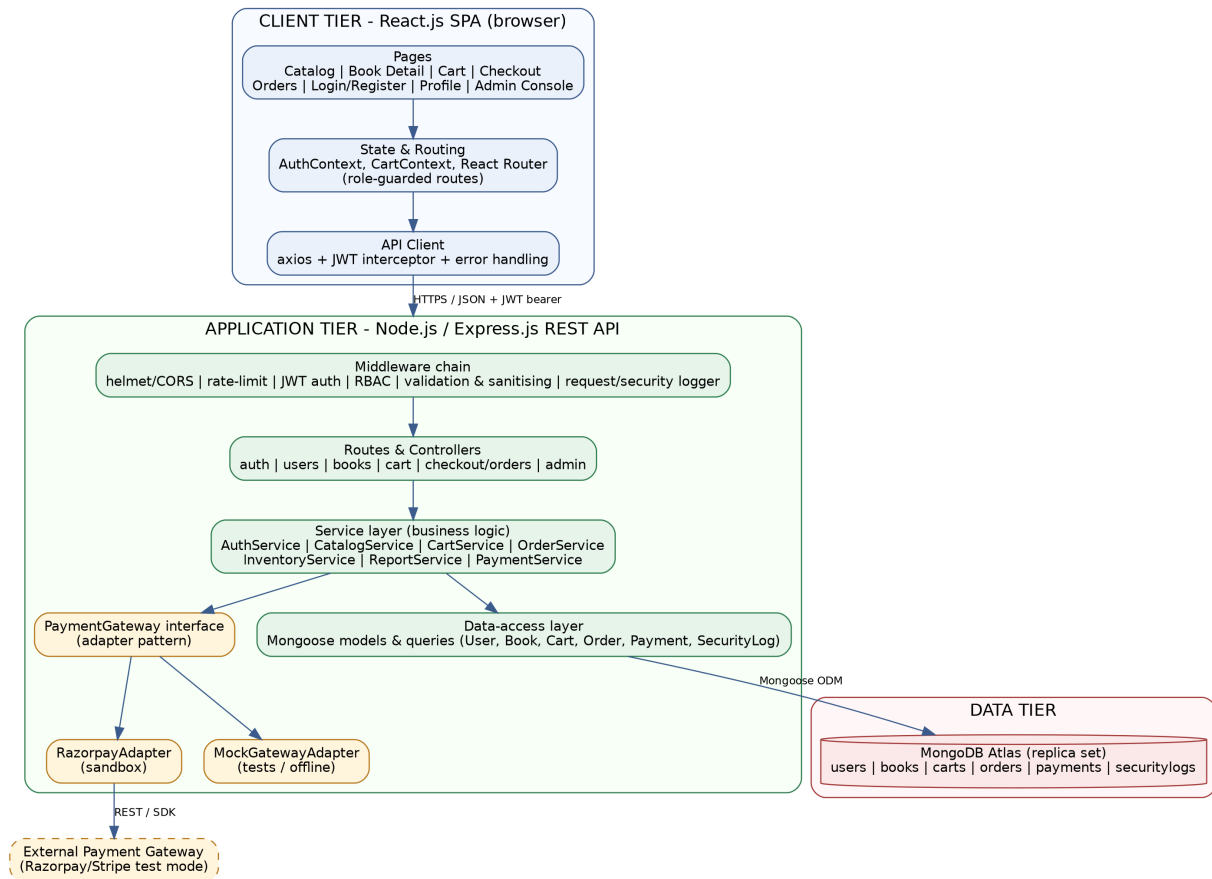


Figure 3.1 – Component diagram: layered architecture of BookNest

3.3.1 Deployment view

The React build is served as static files over HTTPS; the API runs in a Docker container on the cloud host with configuration from environment variables; the database is MongoDB Atlas; GitHub Actions builds, tests, analyses and deploys. A scheduled job takes a daily logical backup (SAF-003).

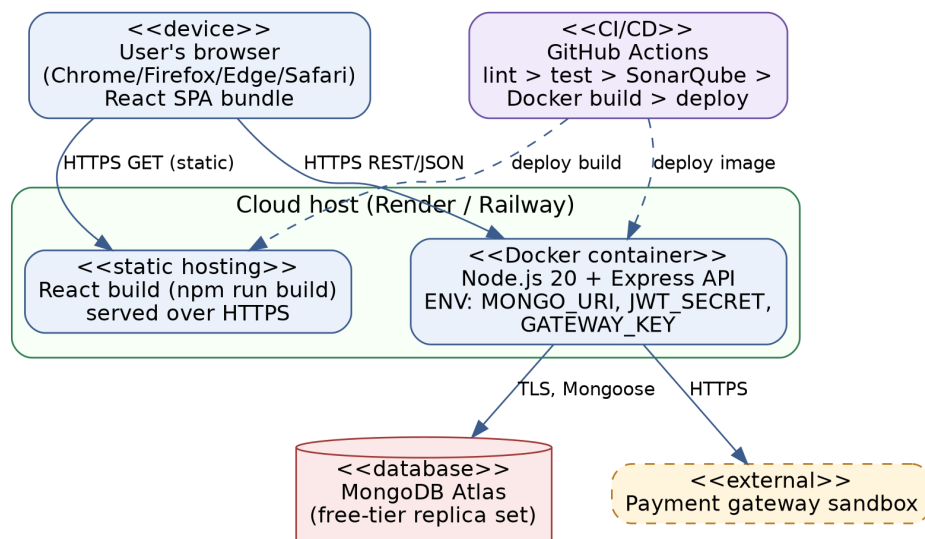


Figure 3.2 – Deployment diagram

3.4 Component Descriptions

Component	Responsibility	Requirements
React pages & routing	Screens for storefront and admin console; role-guarded routes; form validation; loading/error/empty states.	NFR-003, FR-003 to FR-016

Component	Responsibility	Requirements
API client	Adds JWT to requests, sends Idempotency-Key on checkout, maps error responses to user messages.	FR-002, NFR-004
Middleware chain	Security headers, CORS allow-list, rate limiting, JWT verification, role check, schema validation and sanitising, request and security logging.	SEC-002 to SEC-004, SEC-006, SEC-007
AuthService	Register, login (bcrypt compare, JWT issue), profile and password update.	FR-001, FR-002, FR-016, SEC-001
CatalogService	Search, filter, pagination and book detail (customers see only active books).	FR-003, FR-004, FR-015
CartService	Add, update, remove; availability rules; total calculation.	FR-005, FR-006, BR-01
OrderService	Checkout orchestration, order and status queries, forward-only status changes.	FR-007 to FR-010, FR-014, BR-02, BR-04, BR-05, NFR-004, SAF-001
PaymentService + PaymentGateway	Creates and updates Payment records; calls the gateway through an interface; Razorpay-sandbox and Mock adapters.	FR-012, NFR-006, SEC-005
InventoryService	Admin book create/edit/disable, stock adjustment, atomic stock decrement used by checkout.	FR-011, BR-03, BR-06, SAF-002
ReportService	Sales summary (successful payments only) and low-stock list.	FR-013
Data-access layer (Mongoose)	Schemas, validation, indexes for User, Book, Cart, Order, Payment, SecurityLog.	All data requirements
MongoDB	Persistent store; replica set so transactions are available.	7 (SRS)

3.5 Chosen Architecture Pattern and Rationale

BookNest is a **three-tier, layered, modular monolith**: React single-page application, one Express REST API organised as routes/controllers → services → data access, and MongoDB. Layering gives clear separation of concerns (HTTP handling, business rules, persistence), which supports maintainability and unit testing. A single deployable API was chosen on purpose: the team has four members, the user base is one store, and **microservices were rejected** as overly complex – they would add deployment overhead and distributed-consistency problems (orders, payments and stock must change together) with no scaling benefit. The adapter pattern isolates the payment provider. Decisions are recorded below.

ADR	Decision	Alternatives considered	Reason
ADR-01	Layered modular monolith	Microservices	Small team and scale; single deployable API avoids distributed transactions.
ADR-02	Embed CartItem/OrderItem in Cart/Order	Separate collections (as in the ER diagram)	Always accessed with the parent; order creation becomes one atomic write; the ER model stays the logical view.
ADR-03	PaymentGateway interface with adapters	Call the SDK directly	Testing without the live gateway (NFR-006); provider can change without touching OrderService.
ADR-04	Payment-first flow: PENDING payment with unique idempotency key, then order in a transaction	Create order first; no idempotency	Meets FR-007 and NFR-004: no order without payment; retry cannot double-charge.
ADR-05	Stateless JWT (1 h), no sessions, no refresh token	Server sessions; refresh tokens	Keeps API stateless (availability); matches SRS 3.3; refresh flow deferred.
ADR-06	Snapshot price in CartItem/OrderItem; server computes totals	Read live price when displaying	Past orders stay correct after price changes; blocks client tampering.
ADR-07	Search with escaped case-insensitive regex plus compound indexes	MongoDB text index; Atlas Search	Substring match expected by users; about 500 books so cost is negligible; upgrade path if catalog exceeds about 10,000 books.

ADR	Decision	Alternatives considered	Reason
ADR-08	Conditional atomic stock decrement (stock >= qty) in a transaction; refund on failure	Reserve stock at add-to-cart; decrement before payment	BR-06 forbids decrement before payment; conditional update prevents negative stock; refund covers the last-copy race.
ADR-09	Order status as forward-only state machine in OrderService	Free-form status edit	Prevents impossible histories; supports BR-02, FR-014.

3.6 Technology Stack & Data Stores

Concern	Choice (equivalent libraries may be substituted)
Front end	React.js (Vite), React Router, axios, Context API, React Testing Library
API	Node.js 20 LTS, Express.js, Mongoose, jsonwebtoken, bcrypt, Joi/Zod, helmet, express-rate-limit, express-mongo-sanitize, pino
Data store	MongoDB Atlas free-tier replica set (transactions); Decimal128 for money
Payment	Razorpay/Stripe test mode behind PaymentGateway; MockGatewayAdapter for tests
Quality / delivery	Jest, Supertest, Artillery, Postman, SonarQube, GitHub Actions (Jenkins acceptable), Docker, GitHub Projects

Data stores and indexes

Collection	Key fields	Indexes / constraints
users	name, email, password_hash, role, created_at	unique(email, lower-case)
books	title, author, genre, description, price, stock_qty, is_active	(genre, price); (is_active, stock_qty); title and author indexes for prefix search; stock_qty >= 0 validator
carts	customer_id, items[{book_id, quantity, unit_price}], updated_at	unique(customer_id)
orders	order_number, customer_id, items[{book_id, title_snapshot, quantity, unit_price}], status, total, payment_id, payment_status, created_at	unique(order_number); unique(payment_id); (customer_id, created_at desc)
payments	customer_id, order_id (null until SUCCESS), idempotency_key, amount, status, gateway_ref, created_at	unique(idempotency_key); (customer_id, created_at)
securitylogs	timestamp, event, user_id?, ip, route	(timestamp desc); TTL 90 days

Money is stored as Decimal128 (never floating point); order_number is BN-YYYYMMDD-nnnn from an atomic counter, protected by a unique index. The field layouts are in SRS Appendix B and the ER model in SRS Figure 4.2.

3.7 Risks & Mitigations

ID	Risk	Mitigation / owner
R-1	Stock lost between payment authorisation and stock decrement (two buyers, last copy).	Conditional decrement and automatic refund (ADR-08); test TC-ORD-08. M3.
R-2	Payment sandbox unavailable during demo or CI.	MockGatewayAdapter selected by GATEWAY_MODE; CI never calls the live sandbox. M3/M4.
R-3	Free-tier cold starts and small limits distort NFR-001/002 measurements.	Warm-up before load test; record environment in test report; seed about 500 books. M4.
R-4	JWT in browser storage exposed if XSS occurs.	CSP, output escaping, validation, TC-SEC-03; 1-hour expiry. M1.
R-5	MongoDB transactions need a replica set; single-node dev database fails.	Atlas or local replica-set container in docker-compose for dev and CI. M4.
R-6	Data loss (free tier has no managed backup).	Daily mongodump job, 7-day retention, restore test (SAF-003, TC-SAF-01). M4.
R-7	Team member unavailable; uneven contribution.	Each member owns a full functional module (4.1); pull-request reviews by a second member; Git commits checked weekly.

3.8 Traceability to Requirements

Every requirement maps to a component (Section 3.4) and a design element (Section 4). Test case IDs are in the RTM (SRS Appendix C) and the STP.

Req ID	Description	Architecture ref	Design ref
FR-001	Customer registration	SAD 3.4 AuthService	SAD 4.1.2, 4.2.3, 4.3.1
FR-002	Customer login / JWT	SAD 3.4 AuthService; 3.9	SAD 4.2.3, 4.3.1
FR-003	Browse and search catalog	SAD 3.4 CatalogService	SAD 4.1.2, 4.3.2
FR-004	Filter by price and availability	SAD 3.4 CatalogService	SAD 3.6 (indexes), 4.3.2
FR-005	Add, update, remove cart items	SAD 3.4 CartService	SAD 4.1.1, 4.1.2, 4.3.3
FR-006	Cart total calculation	SAD 3.4 CartService	SAD 4.1.1 (Cart.total)
FR-007	Order only after payment authorised	SAD 3.4 OrderService, PaymentService	SAD 4.1.3, 4.2.1, 4.3.4
FR-008	Unique order number	SAD 3.4 OrderService	SAD 3.6 (unique index), 4.2.1
FR-009	View order status	SAD 3.4 OrderService	SAD 4.1.4, 4.3.4
FR-010	Order history	SAD 3.4 OrderService	SAD 4.3.4
FR-011	Admin add/edit/disable books, stock	SAD 3.4 InventoryService	SAD 4.1.2, 4.3.5
FR-012	Record payment success/failure	SAD 3.4 PaymentService	SAD 4.1.3, 4.1.4, 4.2.1
FR-013	Sales summary and low-stock report	SAD 3.4 ReportService	SAD 4.3.6
FR-014	Admin updates order status	SAD 3.4 OrderService	SAD 4.1.4, 4.2.2, 4.3.5
FR-015	Book detail view	SAD 3.4 CatalogService	SAD 4.3.2
FR-016	View/update profile and password	SAD 3.4 AuthService	SAD 4.3.1
NFR-001	Search response time (p95 <= 2 s)	SAD 3.5, 3.6	SAD 3.6 (indexes), ADR-07
NFR-002	Availability >= 99%	SAD 3.3.1, 3.5	SAD 3.3.1 (deployment), 4.4 (health)
NFR-003	Order in <= 5 screens	SAD 3.2	SAD 4.5 (UX)
NFR-004	No duplicate order on retry	SAD 3.5	SAD 4.1.3, 4.2.1, ADR-04
NFR-005	Modular business logic + unit tests	SAD 3.4, 3.5	SAD 4.1.2, ADR-01
NFR-006	Mockable payment adapter	SAD 3.4 PaymentGateway	SAD 4.1.2, ADR-03
SEC-001	Password hashing	SAD 3.9	SAD 3.9 (STRIDE)
SEC-002	RBAC on admin endpoints	SAD 3.9	SAD 4.3.5, 3.9
SEC-003	Input validation and sanitising	SAD 3.9	SAD 3.9, 4.4
SEC-004	Reject expired/invalid JWT; HTTPS	SAD 3.9	SAD 3.9, 4.2.3
SEC-005	No card data stored	SAD 3.9	SAD 4.1.5 (Payment fields), 4.2.1
SEC-006	Security event logging	SAD 3.9	SAD 4.4, 3.6 (securitylogs)
SEC-007	Login rate limiting	SAD 3.9	SAD 3.9
SAF-001	No charge without an order (auto-refund)	SAD 3.4 OrderService, PaymentService	SAD 4.1.3, 4.2.1
SAF-002	Stock never negative	SAD 3.4 InventoryService	SAD 4.1.3, ADR-08
SAF-003	Daily backup and restore	SAD 3.3.1, 3.7	SAD 3.6, 3.7
BR-01	Only available books can be carted/ordered	SAD 3.4 CartService	SAD 4.1.3
BR-02	No order modification after RECEIVED	SAD 3.4 OrderService	SAD 4.1.4
BR-03	Only admin disables books / adjusts stock	SAD 3.9	SAD 4.3.5
BR-04	Order needs at least one valid item	SAD 3.4 OrderService	SAD 4.1.3
BR-05	Payment failure: no order, no stock change	SAD 3.4 OrderService	SAD 4.1.3, 4.2.1
BR-06	Stock decremented only after payment success	SAD 3.4 InventoryService	SAD 4.1.3, 4.2.1

3.9 Security Architecture

Threat modelling (STRIDE) for the main trust boundaries (browser to API, API to database, API to payment gateway):

STRIDE	Threat in BookNest	Control	Req.
Spoofing	Attacker logs in as another user or forges a token; brute-force guessing.	bcrypt-hashed passwords; HS256 JWT signed with environment secret, verified on every request; 5 failed logins per 15 min then 429; identical error for unknown user and wrong password.	SEC-001, SEC-004, SEC-007
Tampering	Client changes price, total, quantity or status; injection of operators into queries; XSS.	Server recomputes totals; schema validation on body/query/path; express-mongo-sanitize; regex-escaped search; output escaping and CSP; forward-only status machine.	SEC-003, FR-006, FR-014
Repudiation	User denies a login attempt or admin action.	securitylogs (timestamp, user, IP, route) for failed logins, 401/403, invalid tokens; request ids in logs.	SEC-006
Information disclosure	Password hashes or card data leak; other users' orders read; stack traces shown.	Hashes never returned; no card data stored (tokenised payment); owner check by JWT user id (404 for others); generic error bodies; HTTPS TLS 1.2+; secrets only in environment.	SEC-001, SEC-005, SEC-004
Denial of service	Request flooding or very large queries.	Rate limiting; pagination (max 50); body-size limit; stateless API with restart policy.	SEC-007, NFR-002
Elevation of privilege	Customer calls admin endpoints or sets own role.	rbac middleware on admin router, role taken only from signed token; profile update ignores role/email; admin routes tested automatically (TC-SEC-01).	SEC-002, BR-03

Trust boundaries and data protection. Card details are entered in the gateway's hosted fields/SDK; BookNest receives only a payment token and stores status and gateway reference. The JWT is kept in browser storage and sent as a bearer token (SRS 3.3); this is exposed if an XSS flaw exists, which is why escaping, CSP and validation are mandatory and tested. CI runs npm audit, secret scanning and SonarQube on every pull request; an OWASP ZAP baseline scan is planned against staging (STP 5.1).

4. Design

4.1 Design Overview

The design follows the layered structure of 3.5. Each business area is a module owned end-to-end by one team member; a module consists of its React pages, routes/controllers, service, model and tests.

Module	Responsibilities	Depends on	Owner
Authentication & profile	Registration, login, JWT, profile, password	User model	M1 Dhanush Gowda A S
Catalog	Search, filter, book detail	Book model	M2 Sahajpreet Singh
Cart	Add/update/remove, total	Catalog service	M3 Rishi Jain
Order & Payment	Checkout, idempotency, order number, history, status view, PaymentGateway adapters	Cart, Inventory, Payment	M3 Rishi Jain
Admin inventory	Book CRUD, enable/disable, stock	Book model	M4 S R Lokesh Adithya
Order processing & Reports	Status updates (FR-014), sales and low-stock reports	Order, Book models	M4 S R Lokesh Adithya

4.1.1 Domain class diagram

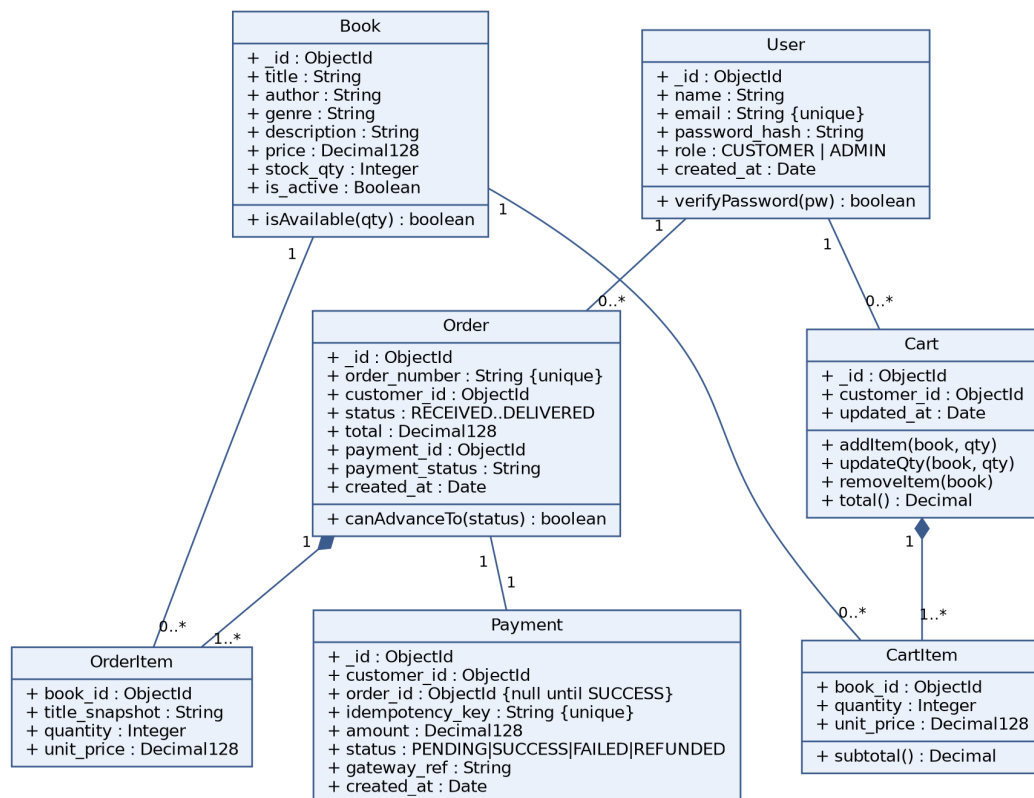


Figure 4.1 – Domain class diagram (multiplicities as in SRS Figure 4.2; diamond = composition)

4.1.2 Service-layer class diagrams

Business-logic classes (Figure 4.2) and the PaymentGateway abstraction (Figure 4.3). OrderService reaches the gateway only through PaymentService and the interface, so the mock can replace Razorpay in tests.

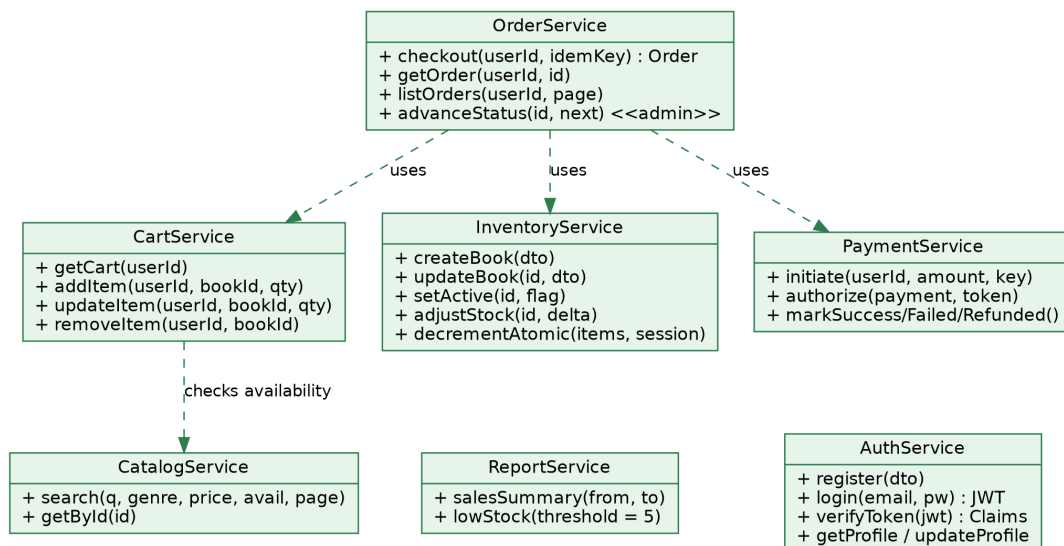


Figure 4.2 – Service-layer class diagram: business services

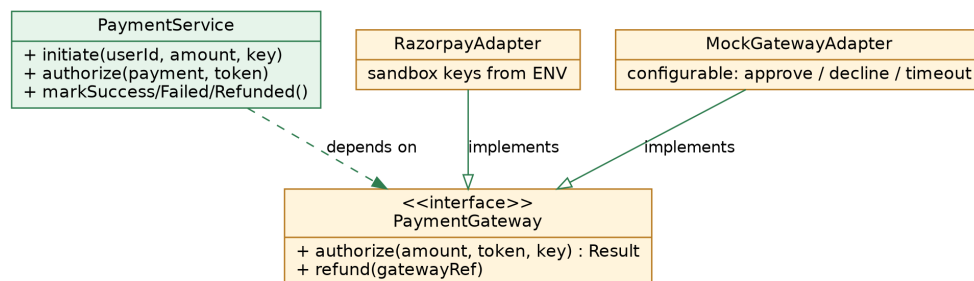


Figure 4.3 – Payment adapter pattern

4.1.3 Activity diagram – Place Order (checkout)

All branches: idempotent replay (NFR-004), cart and stock validation (BR-01, BR-04), payment failure (BR-05) and the stock conflict after a successful payment (SAF-001, SAF-002).

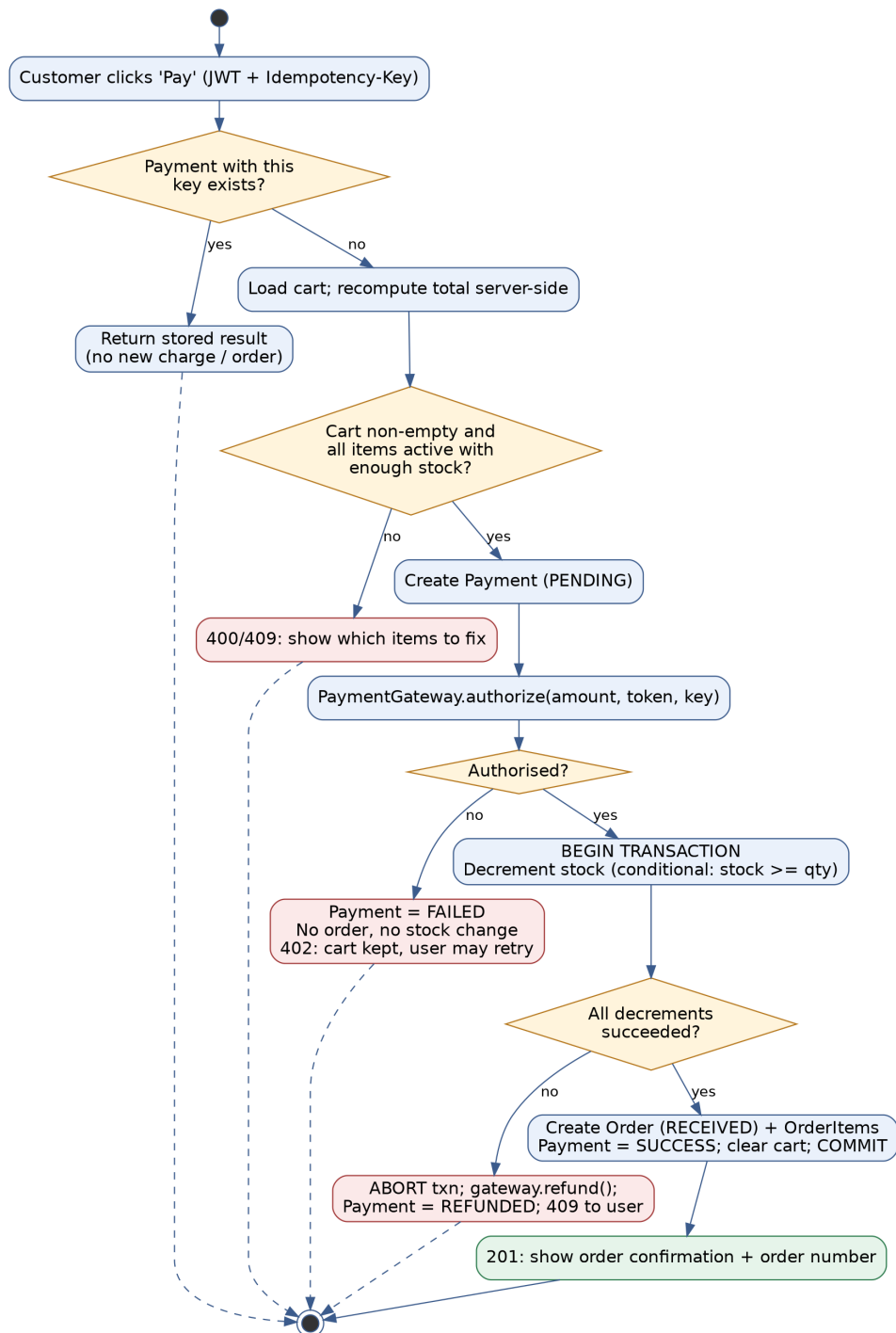


Figure 4.4 – Activity diagram: checkout and order creation

4.1.4 State diagrams – Order and Payment

Order status moves forward one step at a time, only by an administrator (FR-014, DR-01); customers cannot change an order after RECEIVED (BR-02). REFUNDED is used only if stock decrement fails after a successful authorisation.



Figure 4.5 – State diagram: Order.status

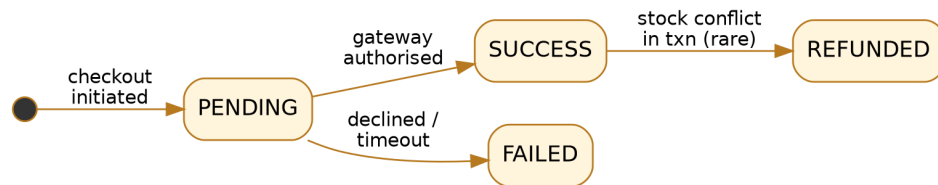


Figure 4.6 – State diagram: *Payment.status*

4.1.5 Data model

The logical model is SRS Figure 4.2; collections and indexes are in 3.6; field layouts are SRS Appendix B.

4.2 UML Sequence Diagrams

Three flows are shown: Place Order (UC-04), Process Order / Update Order Status (UC-10) and Login (UC-01).

4.2.1 Place Order

Card details never reach BookNest: the browser obtains a payment token from the gateway SDK and sends only that token (SEC-005). The server recomputes the total from database prices, records a PENDING payment, authorises it, and only on success creates the order and reduces stock in one transaction.

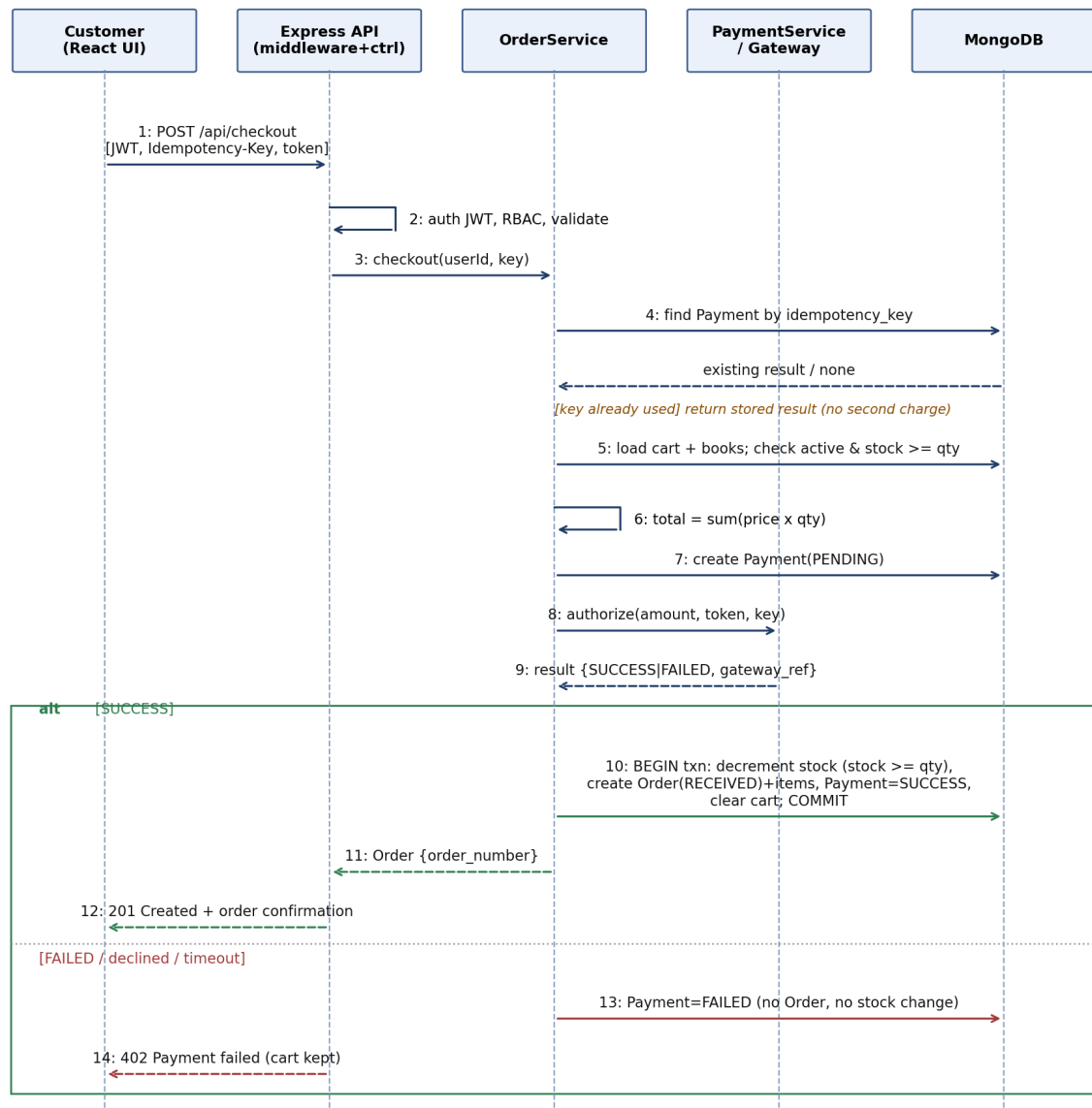


Figure 4.7 – Sequence diagram: Place Order (FR-007, FR-008, FR-012, NFR-004)

4.2.2 Process Order (administrator updates status)

The administrator moves an order one step forward. The update is conditional on the current status, so two simultaneous requests cannot skip a step. The customer sees the new status when the order page is next loaded.

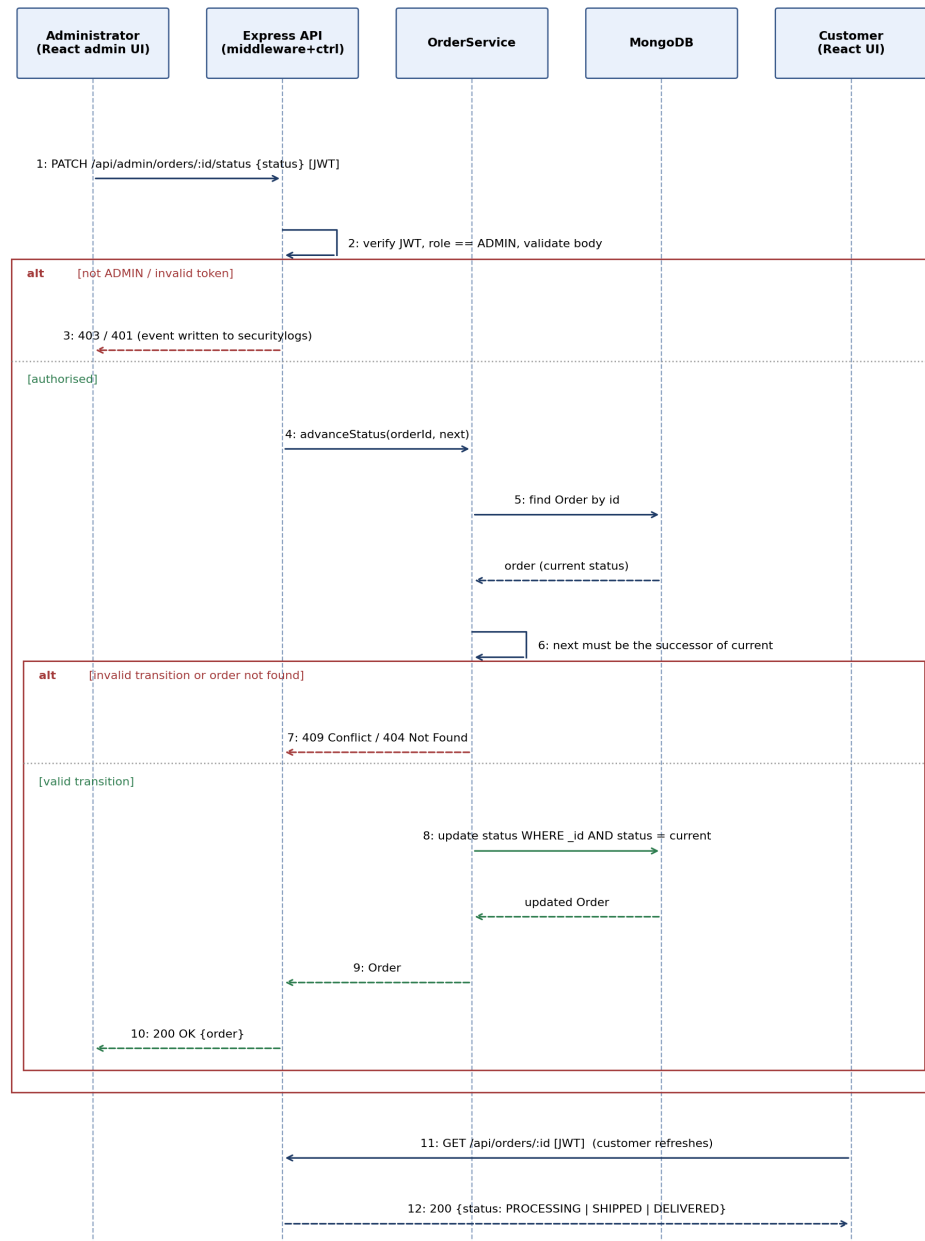


Figure 4.8 – Sequence diagram: Process Order (FR-014, FR-009, SEC-002, BR-02)

4.2.3 Login

Unknown user and wrong password give the same response; failures are logged and rate limited.

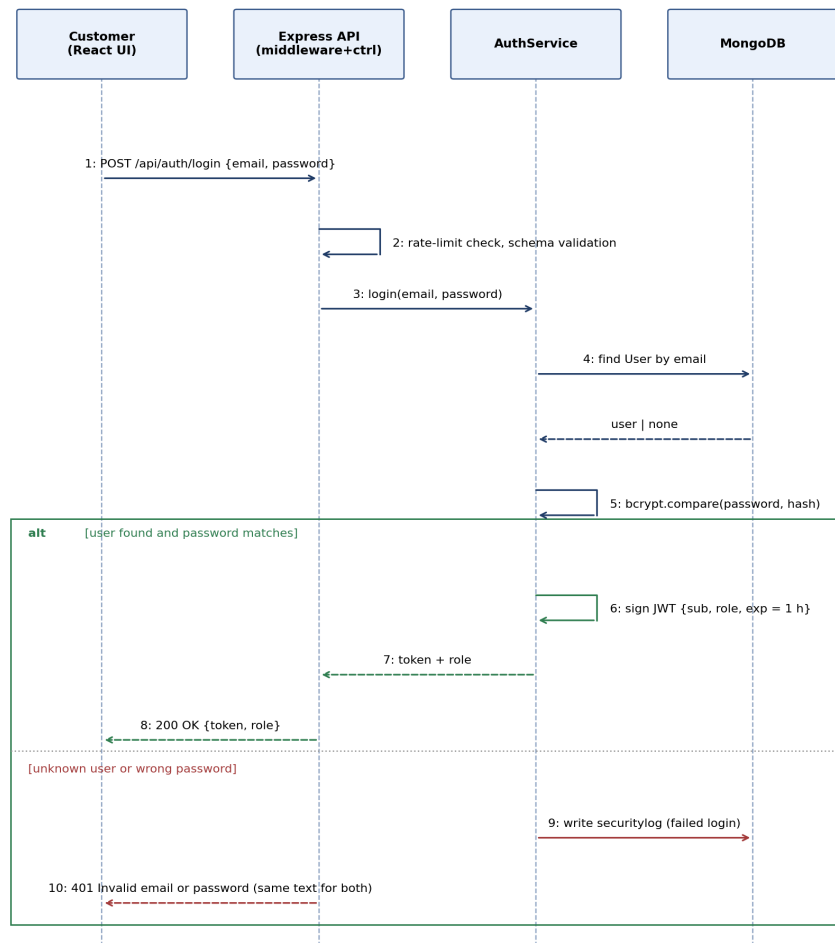


Figure 4.9 – Sequence diagram: Login (FR-002, SEC-001, SEC-004, SEC-006, SEC-007)

4.3 API Design

Contract rules. Every endpoint defines request and response schemas, status codes, authentication/authorisation and validation rules. The base path is `/api` (breaking changes will use `/api/v2`); identifiers are MongoDB ObjectIds; a contract is never changed silently once another module depends on it – changes go through a pull request that updates this section, the Postman collection and the tests. All bodies are JSON. Authenticated calls send `Authorization: Bearer <JWT>`. A resource belonging to a customer is always looked up with the user id in the token, never an id from the request. Page size defaults to 12 (maximum 50).

Common error body: `{ "error": { "code": "STOCK_INSUFFICIENT", "message": "...", "details": [ { "field": "quantity", "issue": "..." } ] } }`. Common statuses: 400 validation, 401 missing/invalid/expired token, 403 role not allowed, 404 not found (also for other users' resources), 409 conflict, 402 payment failed, 429 rate limit.

4.3.1 Auth and profile API (AuthService)

Method / endpoint	Access	Request	Success response	Errors
POST /api/auth/register	Public	{name, email, password}	201 {id, name, email, role}	400 validation; 409 email exists
POST /api/auth/login	Public, rate-limited	{email, password}	200 {token, role, expiresIn}	400; 401 invalid email or password; 429 too many attempts
GET /api/users/me	Logged in	–	200 {id, name, email, role}	401
PUT /api/users/me	Logged in	{name} (role/email ignored)	200 profile	400; 401

Method / endpoint	Access	Request	Success response	Errors
PUT /api/users/me/password	Logged in	{currentPassword, newPassword}	204	400 (same/short password); 401 wrong current password

4.3.2 Catalog API (CatalogService)

Method / endpoint	Access	Request	Success response	Errors
GET /api/books?q&genre;&minPrice;&maxPrice;&available;&page;&limit;	Public	query parameters (validated; operators rejected)	200 {items[{id,title,author,genre,price,inStock}], page, limit, total}	400 invalid parameter
GET /api/books/:id	Public	path id	200 {id,title,author,genre,description,price,inStock}	400 malformed id; 404 unknown/disabled

4.3.3 Cart API (CartService)

Method / endpoint	Access	Request	Success response	Errors
GET /api/cart	Customer	–	200 {items[{bookId,title,quantity,unitPrice,subtotal}], total}	401
POST /api/cart/items	Customer	{bookId, quantity}	200 cart	400; 404 book; 409 unavailable or not enough stock
PATCH /api/cart/items/:bookId	Customer	{quantity}	200 cart	400; 404; 409
DELETE /api/cart/items/:bookId	Customer	–	200 cart	404 not in cart

4.3.4 Checkout and order API (OrderService, PaymentService)

Method / endpoint	Access	Request	Success response	Errors
POST /api/checkout	Customer; header Idempotency-Key (8–64 chars)	{paymentToken}	201 {orderId, orderNumber, status:"RECEIVED", total, paymentStatus:"SUCCESS"}; replay of same key: 200 with same body	400 empty cart / missing key; 402 PAYMENT_DECLINED; 409 item unavailable or stock conflict (payment refunded); 504 gateway timeout (retry with same key)
GET /api/orders?page&limit;	Customer	query	200 {items[{orderId,orderNumber,createdAt,itemCount,total,status}], page, limit, total}	401; 400
GET /api/orders/:id	Customer (owner)	path id	200 {orderNumber, status, items[], total, createdAt}	400; 401; 404 (also for another user's order)

4.3.5 Admin API (InventoryService, OrderService) – role ADMIN only

Method / endpoint	Access	Request	Success response	Errors
POST /api/admin/books	Admin	{title, author, genre?, description?, price, stockQty}	201 book	400; 401; 403
PUT /api/admin/books/:id	Admin	partial book fields	200 book	400; 404
PATCH /api/admin/books/:id/active	Admin	{active: boolean}	200 book	400; 404
PATCH /api/admin/books/:id/stock	Admin	{delta: integer}	200 {stockQty}	400; 404; 409 would be negative
PATCH /api/admin/orders/:id/status	Admin	{status}	200 order	400 unknown status; 404; 409 invalid transition

4.3.6 Reports and health API (ReportService)

Method / endpoint	Access	Request	Success response	Errors
GET /api/admin/reports/sales?from&to;	Admin	dates YYYY-MM-DD	200 {from, to, orderCount, revenue, averageOrderValue}	400 invalid/reversed dates
GET /api/admin/reports/low-stock?threshold=5	Admin	threshold integer (default 5)	200 {threshold, items[{bookId,title,author,genre,stockQty}]}	400

Method / endpoint	Access	Request	Success response	Errors
GET /api/health	Public	–	200 {status:"ok", db:"up"}	503 database unreachable

4.4 Error Handling, Logging & Monitoring

Error handling. Services throw typed errors (ValidationError, NotFound, Conflict, PaymentFailed, Forbidden); a single error-handler middleware maps them to HTTP status and the standard error body. Stack traces and internal messages are never returned. Validation failures list every bad field. Payment errors keep the cart intact so the customer can retry. The React client shows an error banner with the server message and a retry action; forms show inline field messages.

Logging. Structured JSON logs (pino) with request id, route, status and latency. Security events (failed login, 401/403, invalid token, rate-limit hits) are also stored in securitylogs. Passwords, tokens, payment tokens and card data are never logged (SEC-005, SEC-006).

Monitoring. GET /api/health is polled every 5 minutes by an uptime monitor (NFR-002). Metrics reviewed each sprint: checkout failure rate, payment decline rate, 5xx rate, p95 catalog search latency (NFR-001), login failures and 429 count. Alert thresholds (proposed): 5xx above 2% over 15 minutes, health check failing twice in a row. CI results (tests, coverage, SonarQube) are the build-time monitor.

4.5 UX Design

Screen	User	Purpose and key elements	Req.
Login / Register	Visitor	Forms with inline validation, password rules shown, link between the two	FR-001, FR-002
Catalog	Visitor, Customer	Search box, genre and price filters, availability toggle, paged cards (title, author, price, stock badge), cart icon with count	FR-003, FR-004
Book Detail	Visitor, Customer	Full details, description, price, availability, Add to Cart	FR-015, FR-005
Cart	Customer	Lines with quantity stepper and remove, total, Proceed to Checkout	FR-005, FR-006
Checkout	Customer	Order summary, payment fields from the gateway SDK, Pay button disabled while processing, clear success/failure message	FR-007, FR-012
Confirmation / Order Detail	Customer	Order number, items, total, status timeline (RECEIVED to DELIVERED)	FR-008, FR-009
My Orders	Customer	Paged list, newest first	FR-010
Profile	Customer	Change name and password	FR-016
Admin: Books	Administrator	Table with search, add/edit form, enable/disable switch, stock adjust	FR-011
Admin: Orders	Administrator	Order list with status filter and 'advance status' action	FR-014
Admin: Reports	Administrator	Date-range sales summary and low-stock table	FR-013

Purchase path (NFR-003): Catalog → Book Detail → Cart → Checkout (with login/register on first use) → Confirmation: five primary screens. **Principles:** responsive layout (mobile first, breakpoints at 600 and 1024 px); keyboard-operable controls and visible focus; labels on every field; text contrast at least 4.5:1 (WCAG 2.1 AA target); errors and statuses conveyed by text and icons, not colour alone; loading, empty and error states on every data screen; admin routes hidden from customers in the UI and enforced on the server.

4.6 Open Issues & Next Steps

- Confirm with the stakeholder: operating window (08:00–22:00 IST), low-stock threshold (5), INR as the only currency, record retention (3 years).
- Future enhancements: ratings/reviews, order cancellation and refunds, e-mail notifications, refresh tokens, Atlas Search for large catalogs, delivery tracking.
- Replace planned code paths in the RTM with actual file paths as modules are merged; update UML if the implementation diverges.

- Run the OWASP ZAP baseline scan and the Artillery load test on staging in Sprint 5 and record results in the validation report.

5. Appendices

5.1 Glossary

MERN, REST, JWT, RBAC, ODM, ADR, STRIDE, Idempotency key, Txn: see Section 1.4. Catalog, Cart, Order, SKU, Payment Gateway, Visitor: see SRS Appendix A. TLS – Transport Layer Security; CSP – Content Security Policy; CI/CD – continuous integration/delivery; WCAG – Web Content Accessibility Guidelines.

5.2 References

- BookNest SRS v1.1; BookNest STP v1.0; BookNest Synopsis.
- IEEE 42010 / IEEE 1016 (architecture and design descriptions); OWASP Top 10 and ASVS; NIST SP 800-160.
- MongoDB documentation: transactions, indexes; Express and Mongoose documentation; Razorpay/Stripe test-mode documentation.

5.3 Tools

Graphviz/PlantUML-style and matplotlib for diagrams (Draw.io or StarUML acceptable for edits); Postman/Swagger for API documentation; GitHub Projects, GitHub Actions, SonarQube, Docker.



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

Software Test Plan (STP)

BookNest – Online Bookstore Ordering and Inventory Management System

Field	Entry
Project	BookNest – Online Bookstore Ordering and Inventory Management System
Version	1.0
Authors	Team 6 QA (Dhanush Gowda A S, Sahajpreet Singh, Rishi Jain, S R Lokesh Adithya)
Date	1 October 2026
Status	Final for Part-1 submission (test execution results to be recorded in the Actual/Result columns)
Course	UE24CS341A – Software Engineering (Semester V)
Technology Stack	MERN (MongoDB, Express.js, React.js, Node.js)
Customer / Stakeholder	PageTurner Books (Campus Bookstore Manager)

Contents

Contents	2
1. Introduction	3
2. Test Items	3
3. Features to be Tested	3
4. Features Not to be Tested	4
5. Test Approach / Strategy	4
5.1 Security Validation	4
6. Test Environment	5
7. Test Schedule	5
8. Test Deliverables	6
9. Roles and Responsibilities	6
10. Risks and Mitigation	6
11. Assumptions & Dependencies	7
12. Suspension & Resumption Criteria	7
13. Test Case Management & Traceability	7
13.1 Test case ID convention	7
13.2 Coverage by use case / module	7
13.3 Test Case Specifications	9
14. Test Metrics & Reporting	17
15. Approvals	17

1. Introduction

Purpose: This document defines the test plan for BookNest v1.0. It outlines objectives, scope, strategy, environment, schedule, responsibilities and the test cases that verify every requirement of the SRS v1.1 and the design in the SAD v1.0.

Scope: Testing covers customer features (registration/login/profile, catalog search and details, cart, checkout and payment, order tracking and history), administrator features (catalog and inventory management, order status updates, reports), and the non-functional, safety and security requirements. The payment gateway is replaced by a mock adapter, except for one manual sandbox run. Gateway internals, cloud-provider infrastructure and browser engine behaviour are excluded.

References: BookNest SRS v1.1 (requirements and Appendix C traceability matrix); BookNest SAD v1.0; PES University SE project guidelines 2026 and Part-1 deliverables; IEEE 829 test documentation practice; OWASP ASVS/Top 10.

Definitions: SRS – Software Requirements Specification; SAD – Software Architecture and Design Specification; RTM – Requirements Traceability Matrix; TC – test case; ST – system test; JWT – JSON Web Token; RBAC – role-based access control; p95 – 95th percentile; UAT – user acceptance test.

2. Test Items

- Authentication and profile module (register, login, JWT, profile, password)
- Catalog module (search, filter, book detail)
- Cart module
- Order and payment module (checkout, idempotency, PaymentGateway adapters, order tracking and history)
- Admin inventory module (books, stock)
- Order processing module (status updates)
- Reporting module (sales summary, low-stock)
- Cross-cutting: middleware (auth, RBAC, validation, rate limit, logging), React UI, deployment (Docker, CI/CD)

3. Features to be Tested

Features are mapped to SRS requirement IDs (full wording in the SRS):

ID	Feature / requirement	ID	Feature / requirement
FR-001	Customer registration	NFR-004	No duplicate order on retry
FR-002	Customer login / JWT	NFR-005	Modular business logic + unit tests
FR-003	Browse and search catalog	NFR-006	Mockable payment adapter
FR-004	Filter by price and availability	SEC-001	Password hashing
FR-005	Add, update, remove cart items	SEC-002	RBAC on admin endpoints
FR-006	Cart total calculation	SEC-003	Input validation and sanitising
FR-007	Order only after payment authorised	SEC-004	Reject expired/invalid JWT; HTTPS
FR-008	Unique order number	SEC-005	No card data stored
FR-009	View order status	SEC-006	Security event logging
FR-010	Order history	SEC-007	Login rate limiting
FR-011	Admin add/edit/disable books, stock	SAF-001	No charge without an order (auto-refund)
FR-012	Record payment success/failure	SAF-002	Stock never negative
FR-013	Sales summary and low-stock report	SAF-003	Daily backup and restore
FR-014	Admin updates order status	BR-01	Only available books can be carted/ordered
FR-015	Book detail view	BR-02	No order modification after RECEIVED

ID	Feature / requirement	ID	Feature / requirement
FR-016	View/update profile and password	BR-03	Only admin disables books / adjusts stock
NFR-001	Search response time (p95 <= 2 s)	BR-04	Order needs at least one valid item
NFR-002	Availability >= 99%	BR-05	Payment failure: no order, no stock change
NFR-003	Order in <= 5 screens	BR-06	Stock decremented only after payment success

4. Features Not to be Tested

- Internal behaviour of the payment provider (sandbox is treated as a black box; only the adapter contract is tested).
- Cloud host and MongoDB Atlas platform internals (vendor responsibility).
- Browser engine rendering differences beyond a smoke test on the latest Chrome, Firefox and Edge.
- Features outside release 1.0: ratings/reviews, cancellation and returns, notifications, multiple currencies.
- Load beyond 50 concurrent users and full-scale penetration testing (planning only at this stage).

5. Test Approach / Strategy

Levels

Level	Purpose	Tools	Owner	When
Unit	Business rules in isolation: cart total, stock decrement, status transitions, hashing, validation schemas	Jest with mocked models and gateway	Module owner	Every commit (CI)
Component (UI)	React screens: loading, error, form validation, role-guarded routes	React Testing Library	M2 (storefront), M4 (admin)	Every commit (CI)
Integration	API with a real test MongoDB replica set and MockGatewayAdapter: status codes, persistence, RBAC, transactions	Jest + Supertest, Postman	Module owner	Every pull request (CI)
System	End-to-end journeys ST-01 to ST-08 on the deployed system	Manual scripted runs, Postman	M4 lead; all execute	End of Sprints 3, 5, 6
Acceptance (UAT)	Stakeholder confirms behaviour	Demo of ST-02, ST-05, ST-06	All	Sprint 6

Types

- **Functional testing** (core features) – black-box with equivalence partitioning and boundary values; positive and negative case for every High-priority requirement.
- **Regression testing** – the full unit and integration suite runs on every pull request; failures block merging.
- **Performance testing** – TC-PERF-01 (50 users, p95 ≤ 2 s) with Artillery.
- **Usability testing** – TC-USE-01 task walkthrough with 5 first-time users; accessibility checks on key screens.
- **Reliability and safety testing** – payment timeout/retry, concurrent last-copy purchase, backup restore.
- **Security testing** – see 5.1.
- **White-box measures** – line coverage of the services layer at least 80% and branch coverage reported (target 70%); every branch of OrderService.checkout exercised.

Entry criteria: stable build delivered and deployed to the test environment; test data seeded; test cases reviewed; environment available. **Exit criteria:** 100% of planned test cases executed and results recorded; 100% of High-priority test cases passed; no Critical or High defect open (at most 3 Medium with a workaround); services line coverage ≥ 80% and branch coverage reported; SonarQube quality gate passed; NFR-001 result recorded; all acceptance criteria satisfied.

5.1 Security Validation

Security area	Validation activity	Test case(s)	Req.
Password handling	Bcrypt hash inspection; no plaintext in DB or logs; masked password fields in UI	TC-AUTH-06, TC-AUTH-13	SEC-001
Authentication and token handling	Expired, tampered and missing JWT; secrets only in environment; rate limiting of login	TC-AUTH-07, 08, 09, TC-TRK-04	SEC-004, SEC-007
Authorization (RBAC)	Customer and anonymous calls to every admin route; ownership checks (cart, orders)	TC-SEC-01, TC-ADM-08, TC-OPS-05, TC-RPT-08, TC-CART-08, TC-TRK-02	SEC-002, BR-03
Input validation / fuzzing	Invalid, oversized and operator-injection input on registration, login, search, admin forms, status and report parameters; XSS payloads	TC-AUTH-03, TC-CAT-08, TC-SEC-03, TC-SEC-05, TC-ADM-02, TC-OPS-06, TC-RPT-05	SEC-003
Payment data	No card number/CVV in DB, logs or responses; server-side price and total	TC-SEC-04, TC-ORD-09	SEC-005
Security logging	Failed login, 403 and invalid-token events recorded without secrets	TC-SEC-02, TC-AUTH-05	SEC-006
Transport security	HTTP redirects to HTTPS; TLS 1.2+ only; security headers (helmet)	TC-SEC-06	SEC-004
Static analysis / dependencies	SonarQube security hotspots reviewed; npm audit shows no High/Critical	TC-SEC-07	SEC-003
Penetration testing (planned)	OWASP ZAP baseline scan and manual checks of login, cart, checkout and admin paths on staging; findings logged as defects with remediation	TC-SEC-08	SEC-002, SEC-003

6. Test Environment

Hardware: developer laptops (8 GB RAM minimum); GitHub Actions Ubuntu runners; cloud host for staging (free tier); client devices – one desktop browser and one Android/iOS phone for responsive checks.

Software: BookNest v1.0 (React front end, Express API, Node.js 20), MongoDB replica set (local Docker Compose for dev/CI; Atlas for staging), mock payment gateway (Razorpay sandbox for one manual run), Chrome/Firefox/Edge latest.

Tools: Jest, Supertest, React Testing Library (automation); Postman (API collection); Artillery (performance); OWASP ZAP (security scan); SonarQube and npm audit (static analysis); GitHub Actions (CI); GitHub Issues/Projects (defect tracking and Agile board); uptime monitor (availability).

Test data: seed script with 500 books in 8 genres including boundary prices 199.99, 200.00, 500.00, 500.01, stock 0, 1, 3, 5, 6 and two disabled books; 1 administrator and 5 customers (one with 12 orders); mock tokens tok_success, tok_decline, tok_timeout; no real card data. The database is reset before every integration run and before each system or performance run.

7. Test Schedule

Milestones follow the six Agile sprints of the Synopsis; calendar dates are those of the team's sprint calendar.

Milestone	Sprint	Test activity	Evidence
Test plan and test case design	With this submission (due 3 Oct 2026)	Plan, test cases and traceability reviewed by all members	This document
Environment setup	Sprint 1	Docker Compose replica set, CI pipeline, seed script, mock gateway	Green pipeline run
Test execution	Sprint 1	TC-AUTH-01 to 17	Jest report, Postman collection
	Sprint 2	TC-CAT-01 to 09	Jest/RTL report, demo
	Sprint 3	TC-CART-*, TC-ORD-*, ST-02, ST-03, ST-04	Integration test report, PRs with reviews

Milestone	Sprint	Test activity	Evidence
	Sprint 4	TC-TRK-*, TC-OPS-*, ST-06	System demo, test evidence
	Sprint 5	TC-ADM-*, TC-RPT-*, TC-SEC-*, TC-PERF-01, TC-MNT-01, TC-SAF-01, ST-05, ST-07, ST-08	Coverage, SonarQube, load-test and ZAP reports
UAT and release	Sprint 6	Full regression, TC-AVL-01, TC-USE-01, all ST-*, stakeholder demo	Final test summary report

8. Test Deliverables

- Test Plan (this document)
- Test cases (this document, Section 13.3, and test-cases spreadsheet in the repository)
- Test scripts (Jest, Supertest, RTL, Postman collection, Artillery configuration)
- Test data (seed script, mock tokens)
- Test execution logs (Actual/Result columns, CI logs)
- Defect reports (GitHub Issues)
- Unit test and coverage reports (line and branch)
- Security validation evidence (ZAP report, SonarQube report, npm audit)
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	S R Lokesh Adithya (M4)	Prepares plan, owns CI pipeline and SonarQube, security and performance tests, coordinates execution, consolidates test report
Test Engineer – Authentication	Dhanush Gowda A S (M1)	Designs and executes TC-AUTH-*, JWT/password security checks
Test Engineer – Catalog and UI	Sahajpreet Singh (M2)	Designs and executes TC-CAT-*, UI component tests, usability walkthrough
Test Engineer – Cart, Order, Payment	Rishi Jain (M3)	Designs and executes TC-CART-*, TC-ORD-*, TC-TRK-*; maintains seed script and mock gateway
Developers	All members	Fix defects, write unit tests for own module, review each other's pull requests
Product Owner	Pageturner Books manager / course instructor	Approves test results and sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build from a module owner	Early smoke builds each sprint; CI keeps main always buildable
Staging or CI environment downtime	Docker Compose environment can be recreated locally; seed script restores data
Payment sandbox unavailable	MockGatewayAdapter for all automated tests; sandbox used once manually
Free-tier cold starts distort performance results	Warm-up requests before measuring; record environment in the report
Concurrency tests are flaky	Use fixed seeds and repeat runs (5 times); assertions on final database state, not timing
Uneven test effort among members	Each member owns test cases for own module; pull requests need tests; QA lead tracks execution status

11. Assumptions & Dependencies

- MongoDB replica set (local or Atlas) is available so transactions can be tested.
- Test data and mock tokens are provided by the seed script before execution.
- The sandbox gateway account remains active for the manual run.
- Five first-time users outside the team are available for the usability walkthrough.
- Operating window (08:00–22:00 IST), 50-user load and 500-book catalog are as stated in the SRS.

12. Suspension & Resumption Criteria

Suspend testing if: the environment is unavailable for more than 4 hours; the build does not start; a Critical defect blocks more than 30% of remaining test cases.

Resume testing if: blocking defects are resolved, the build is redeployed and the smoke tests (TC-AUTH-04, TC-CAT-01, TC-ORD-01) pass.

13. Test Case Management & Traceability

The RTM (SRS Appendix C) maps each requirement to architecture, design, code, test case and system test. Examples: FR-001 (registration) → TC-AUTH-01, TC-AUTH-02, TC-AUTH-03; FR-007 (order after payment) → TC-ORD-01, TC-ORD-02, ST-02, ST-03; SEC-002 (RBAC) → TC-SEC-01, TC-ADM-08, ST-07; NFR-001 (search time) → TC-PERF-01. Each test case below lists the requirement(s) it verifies, so the mapping can be checked in both directions. When a feature is merged, its author replaces the planned code path in the RTM with the real file path and adds the result; the QA lead checks the matrix before each sprint review.

13.1 Test case ID convention

TC-<MODULE>-nn for unit, component, integration, security, performance and other tests; ST-nn for system (end-to-end) tests. Levels: Unit, Component, Integration, System, Security, Performance, Operational, Review.

13.2 Coverage by use case / module

Module	Use case / group	Test cases	Unit / Component	Integration	System / other
Auth (UC-01)	UC-01	9	2	6	1
Auth (UC-09)	UC-09	8	2	6	0
Catalog (UC-02)	UC-02	9	1	8	0
Cart (UC-03)	UC-03	8	2	6	0
Order (UC-04)	UC-04	11	2	9	0
Tracking (UC-05)	UC-05 (+UC-08)	9	0	8	1
Order processing (UC-10)	UC-10	8	0	8	0
Admin catalog (UC-06)	UC-06	8	0	8	0
Reports (UC-07)	UC-07	8	0	8	0
Security	SEC-*	8	0	4	4
Non-functional	NFR-*, SAF-003	5	0	0	5
System (E2E)	all	8	0	0	8
Total		99			

Every implemented use case has at least 8 test cases (guideline: 8–10 per use case), covering unit, integration and system levels; security and non-functional requirements have their own cases (modules Security and Non-functional).

Requirement group	Requirements	Covered	Coverage
Functional (FR)	16	16	100%

Requirement group	Requirements	Covered	Coverage
Non-functional (NFR)	6	6	100%
Safety (SAF)	3	3	100%
Security (SEC)	7	7	100%
Business rules (BR)	6	6	100%
Total	38	38	100%

13.3 Test Case Specifications

Actual and Result are completed after manual or automated execution (Pass/Fail) and kept in the repository test-results folder; they are intentionally blank in this planning version.

Auth (UC-01)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-AUTH-01	FR-001	Integration / Functional	Register with valid data	Email not yet registered	POST /api/auth/register	name=Asha Rao; email=asha@test.in; password=Book#2026	201 Created; user stored with role CUSTOMER; response contains no password data		
TC-AUTH-02	FR-001	Integration / Negative	Register with an existing email	asha@test.in already registered	POST /api/auth/register twice (second with ASHA@test.in)	same data; email in upper case	Second call returns 409; only one user document exists		
TC-AUTH-03	FR-001, SEC-003	Unit + Integration / Negative	Registration field validation	API running	Submit each invalid body in turn	empty name; email=abc; password of 7 chars; name of 1000 chars	400 with a message per field; nothing stored; UI shows the same messages		
TC-AUTH-04	FR-002	Integration / Functional	Login with valid credentials	asha@test.in registered	POST /api/auth/login	asha@test.in / Book#2026	200 and signed JWT (sub, role, exp); token accepted by a protected route		
TC-AUTH-05	FR-002, SEC-006	Integration / Negative	Login with wrong password or unknown email	asha@test.in registered	POST /api/auth/login twice	Wrong#123; nobody@test.in	401 with identical message for both (no user enumeration); both failures logged in securitylogs		
TC-AUTH-06	SEC-001	Unit + Review / Security	Password stored only as a hash	Two users registered with the same password	Read stored documents; run hash-utility unit test	password=Book#2026 for both	password_hash is bcrypt (cost >= 10), not equal to plaintext, different for the two users		
TC-AUTH-07	SEC-004	Integration / Security	Expired, tampered and missing JWT	Valid user exists	GET /api/orders with each token variant	expired token; token with altered payload; no Authorization header	401 in all three cases; no data returned		
TC-AUTH-08	SEC-004	Review / Security	Secrets are not in source control	Repository and CI logs available	Run secret scan; inspect config loader	JWT_SECRET, MONGO_URI, GATEWAY_KEY	Secrets read only from environment variables; scan finds nothing		
TC-AUTH-09	SEC-007	Integration / Security	Login rate limiting	Account exists	Send 6 failed logins within 15 minutes, then a correct login	Wrong#123 x 6	6th attempt returns 429; correct login succeeds after the window		

Auth (UC-09)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-AUTH-10	FR-016	Integration / Functional	View and update profile	Logged-in customer	GET then PUT /api/users/me	name=Asha R; body also contains role=ADMIN and email=x@y.in	Name updated; role and email unchanged (mass assignment blocked)		
TC-AUTH-11	FR-016	Integration / Functional	Change password	Logged-in customer	PUT /api/users/me/password	current=Book#2026; new=New#Pass2026	200; old password no longer logs in; new one does		
TC-AUTH-12	FR-016, SEC-003	Integration / Negative	Change password with invalid input	Logged-in customer	PUT /api/users/me/password	wrong current password; new password=abc	401/400 with message; password unchanged		

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-AUTH-13	FR-016, SEC-001	Integration / Security	Profile response hides password data	Logged-in customer	GET /api/users/me	valid JWT	200 with name, email, role only; no password_hash field		
TC-AUTH-14	FR-016, SEC-004	Integration / Security	Profile without login	No token	GET and PUT /api/users/me	no Authorization header	401 for both; nothing changed		
TC-AUTH-15	FR-016, SEC-003	Unit + Integration / Boundary	Name length limits	Logged-in customer	PUT /api/users/me with each name	empty name; 100 chars; 101 chars	Empty rejected (400); 100 accepted; 101 rejected (400)		
TC-AUTH-16	FR-016	Integration / Negative	New password equal to current	Logged-in customer	PUT /api/users/me/password	current=Book#2026; new=Book#2026	400 with message; password unchanged		
TC-AUTH-17	FR-016	Unit / Functional	Name is trimmed before saving	Logged-in customer	PUT /api/users/me	name=' Asha R '	Stored as 'Asha R'		

Catalog (UC-02)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-CAT-01	FR-003	Integration / Functional	Search by title (visitor)	500 books seeded; no login	GET /api/books?q=harry	q=harry	200; only books whose title contains the text (case-insensitive)		
TC-CAT-02	FR-003	Integration / Functional	Search by author	Books seeded	GET /api/books?q=rowling	q=rowling	Books by matching authors returned		
TC-CAT-03	FR-003	Integration / Functional	Filter by genre	Books seeded in 8 genres	GET /api/books?genre=Fiction	genre=Fiction	Only Fiction titles; paginated (12 per page)		
TC-CAT-04	FR-003	Integration + UI / Negative	Search with no match	Books seeded	GET /api/books?q=zzzzzz	q=zzzzzz	200 with empty list; UI shows a 'no results' message, not an error		
TC-CAT-05	FR-004	Integration / Boundary	Price range with inclusive bounds	Books priced 199.99, 200.00, 500.00, 500.01	GET /api/books?minPrice=200&maxPrice=500	minPrice=200; maxPrice=500	200.00, 500.00 and values between are returned; 199.99 and 500.01 are not		
TC-CAT-06	FR-004	Integration / Functional	Availability filter	Books with stock 0 and disabled books exist	GET /api/books?available=true	available=true	Out-of-stock and disabled books are excluded		
TC-CAT-07	FR-015	Integration + UI / Functional	Book detail view	One active and one disabled book	GET /api/books/:id for active, unknown and disabled ids	valid id; 00000000000000000000000000; disabled id	Active: title, author, genre, description, price, availability shown. Unknown/disabled: 404		
TC-CAT-08	SEC-003	Integration / Security	Query-injection attempt	Books seeded	GET /api/books with operator and regex input	q[\$ne]=x; q=.*; genre={"\$gt":"."}	Operators rejected (400) or treated as literal text; no data leak; server stays up		
TC-CAT-09	FR-003, NFR-003	Component (RTL) / Functional	Loading, error and validation states	React app with mocked API	Delay API; make API fail; submit invalid form	API delay 3 s; HTTP 500; empty required field	Spinner while loading; friendly error on failure; inline validation messages		

Cart (UC-03)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-CART-01	FR-005, BR-01	Integration / Functional	Add an available book	Logged in; book stock 10	POST /api/cart/items	bookId=B1; quantity=1	Item added with unit_price copied from the book; total updated		

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-CART-02	FR-005	Integration / Functional	Add the same book again	B1 already in cart	POST /api/cart/items again	bookId=B1; quantity=2	One line with quantity 3; no duplicate line		
TC-CART-03	FR-005, BR-01	Integration / Negative	Add out-of-stock or disabled book	Book with stock 0; disabled book	POST /api/cart/items for each	stock=0 book; disabled book	409/400 returned; cart unchanged		
TC-CART-04	FR-005	Integration / Boundary	Update quantity at limits	B1 in cart; stock 5	PATCH /api/cart/items/B1	quantity = 5, 6, 0, -1, 2.5	5 accepted; 6 rejected (409); 0, negative and non-integer rejected (400)		
TC-CART-05	FR-005	Integration / Functional	Remove an item	B1 in cart	DELETE /api/cart/items/B1, then again	bookId=B1	Line removed and total recalculated; second delete returns 404		
TC-CART-06	FR-006	Unit / Functional	Cart total calculation	Cart object in memory	Call total() with 2 lines	2 x 199.99 + 1 x 349.50	Exactly 749.48 (no floating-point drift)		
TC-CART-07	FR-006	Unit / Boundary	Empty cart and large quantity	Cart object in memory	Call total() on empty cart and on qty 999	empty; 999 x 199.99	0.00; 199790.01 computed exactly		
TC-CART-08	SEC-002	Integration / Security	Cart ownership	Two customers A and B with carts	B calls cart endpoints with A's item ids	A's bookId in B's request	Cart is resolved from the JWT user only; A's cart is never read or changed		

Order (UC-04)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-ORD-01	FR-007, FR-008, FR-012, BR-06	Integration / Functional	Successful checkout	Cart with 2 items; mock gateway approves	POST /api/checkout with Idempotency-Key	token=tok_success; key=K1	201; Order RECEIVED with unique order_number; Payment SUCCESS with gateway_ref; stock reduced; cart cleared		
TC-ORD-02	FR-007, FR-012, BR-05	Integration / Negative	Payment declined	Cart with items; mock gateway declines	POST /api/checkout	token=tok_decline	402; no Order; Payment FAILED recorded; stock unchanged; cart retained		
TC-ORD-03	BR-04	Integration / Negative	Checkout with empty cart	Logged in; empty cart	POST /api/checkout	empty cart	400; no Payment or Order created; gateway not called		
TC-ORD-04	NFR-004, SAF-001	Integration / Reliability	Retry with the same idempotency key	Successful checkout K1 done	Send the same checkout request again	key=K1	Original order returned; one Order, one Payment, one stock decrement		
TC-ORD-05	NFR-004, SAF-001, FR-007	Integration / Reliability	Gateway timeout then retry	Mock gateway: timeout on first call, approve on second	Checkout; client retries with same key	token=tok_timeout then tok_success	No duplicate order or charge; final state one RECEIVED order (or FAILED payment and no order if retry also fails)		
TC-ORD-06	FR-008	Integration / Functional	Order numbers are unique under load	100 customers with valid carts	Run 100 checkouts concurrently	100 different keys	All order_number values distinct (unique index enforced)		
TC-ORD-07	BR-06, SAF-002	Unit + Integration / Functional	Stock decrement amount	Book stock 5	Order 2 copies; then a failed payment for 2 copies	qty=2	Stock 3 after success; still 5 after the failed payment		
TC-ORD-08	FR-007, BR-05, SAF-001, SAF-002	Integration / Concurrency	Last-copy race	Book stock 1; two customers hold it in cart	Both press Pay at the same time	stock=1; two tokens tok_success	Exactly one order created; the other payment REFUNDED with 409; stock never negative		

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-ORD-09	FR-007, FR-006	Integration / Security	Server-side price and total	Cart with a book priced 199.99	POST /api/checkout with tampered body	body price=1.00; total=1.00	Client amounts ignored; charge equals total recomputed from database prices		
TC-ORD-10	NFR-006	Unit / Testability	Order flow runs without the live gateway	MockGatewayAdapter configured; network disabled	Run OrderService unit tests	GATEWAY_MODE=mock	Tests pass offline; adapter swapped by configuration only		
TC-ORD-11	BR-01, BR-04	Integration / Negative	Book disabled after being carted	Book in cart; admin then disables it	POST /api/checkout	disabled book B1	409 naming the unavailable item; no Payment created		

Tracking (UC-05)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-TRK-01	FR-009	Integration / Functional	View status of own order	Customer has order in RECEIVED	GET /api/orders/:id as owner	orderId=O1	200 with status RECEIVED, items and total		
TC-TRK-02	FR-009, SEC-002	Integration / Security	View another customer's order	Orders O1 (customer A) exists	GET /api/orders/O1 as customer B	orderId=O1	404; no order data returned		
TC-TRK-03	FR-009	Integration / Negative	Malformed or unknown order id	Logged in	GET /api/orders/:id	id=123; id=00000000000000000000000000000000	400 for malformed id; 404 for unknown id		
TC-TRK-04	FR-009, SEC-004	Integration / Security	Track order without login	Order exists	GET /api/orders/O1 without token	no token	401		
TC-TRK-05	FR-009, FR-014	System / Functional	Status reflects admin updates	Order O1 in RECEIVED	Admin sets PROCESSING; customer reloads order page	status=PROCESSING	Customer sees PROCESSING without logging in again		
TC-TRK-06	FR-009, FR-006	Integration / Functional	Order keeps original prices	Order O1 placed at 199.99	Admin changes book price to 250; customer opens O1	new price=250.00	Order still shows unit_price 199.99 and original total		
TC-TRK-07	FR-010	Integration / Functional	Order history, newest first	Customer with 12 orders; another customer with 3	GET /api/orders	page=1 limit=5	5 own orders, newest first; none from the other customer		
TC-TRK-08	FR-010	Integration / Boundary	History pagination limits	Customer with 12 orders	GET /api/orders at page 3 and page 4	limit=5; page=3; page=4	Page 3 has 2 orders; page 4 returns an empty list		
TC-TRK-09	FR-010	Integration + UI / Functional	Empty order history	New customer with no orders	Open My Orders	new account	Empty list and a 'no orders yet' message		

Order processing (UC-10)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-OPS-01	FR-014	Integration / Functional	Advance RECEIVED to PROCESSING	Admin logged in; order RECEIVED	PATCH /api/admin/orders/:id/status	status=PROCESSING	200; order status PROCESSING		
TC-OPS-02	FR-014	Integration / Functional	Complete lifecycle	Order RECEIVED	Advance to PROCESSING, SHIPPED, DELIVERED	three PATCH calls	Every step accepted in order; final status DELIVERED		
TC-OPS-03	FR-014	Integration / Negative	Skip a status	Order RECEIVED	PATCH status=DELIVERED	status=DELIVERED	409; status stays RECEIVED		

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-OPS-04	FR-014	Integration / Negative	Move status backwards	Order SHIPPED	PATCH status=PROCESSING	status=PROCESSING	409; status stays SHIPPED		
TC-OPS-05	FR-014, SEC-002	Integration / Security	Customer tries to change status	Customer token; order RECEIVED	PATCH /api/admin/orders/:id/status	status=PROCESSING	403; status unchanged		
TC-OPS-06	FR-014, SEC-003	Integration / Negative	Invalid status value or unknown order	Admin logged in	PATCH with bad value; PATCH unknown id	status=COMPLETED; id=0 0000000000000000000000000000000000	400 for bad value; 404 for unknown order		
TC-OPS-07	BR-02	Integration / Negative	Customer modifies order after RECEIVED	Order PROCESSING; customer logged in	Customer calls order edit/cancel endpoint	PUT/DELETE /api/orders/:id	403/409; order unchanged		
TC-OPS-08	FR-014	Integration / Concurrency	Two admins advance the same order	Order RECEIVED	Send two PATCH status=PROCESSING simultaneously	two requests	Exactly one succeeds; the other gets 409; no skipped status		

Admin catalog (UC-06)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-ADM-01	FR-011	Integration / Functional	Admin adds a book	Admin logged in	POST /api/admin/books	title=Test Book; author=A B; price=299.00; stock=10	201; book appears in catalog search		
TC-ADM-02	FR-011, SEC-003	Integration / Negative	Invalid book data	Admin logged in	POST /api/admin/books with each bad body	price=-5; missing title; stock=2.5; price=abc	400 with field messages; nothing saved		
TC-ADM-03	FR-011	Integration / Functional	Edit a book	Book B1 exists	PUT /api/admin/books/B1	price 299.00 to 349.00	Saved; new carts use new price; existing orders keep old price		
TC-ADM-04	FR-011, BR-03	Integration / Functional	Increase stock	B1 stock 3	PATCH /api/admin/books/B1/stock	delta=+10	Stock 13		
TC-ADM-05	FR-011, SAF-002	Integration / Boundary	Stock cannot go below zero	B1 stock 3	PATCH stock with delta=-4 and delta=-3	delta=-4; delta=-3	-4 rejected (400/409), stock stays 3; -3 accepted, stock 0		
TC-ADM-06	FR-011, BR-01, BR-03	Integration / Functional	Disable a listing	B1 active and in search results	PATCH /api/admin/books/B1/active	active=false	Hidden from customers; cannot be added to cart; past orders unaffected		
TC-ADM-07	FR-011	Integration / Functional	Re-enable a listing	B1 disabled	PATCH /api/admin/books/B1/active	active=true	Visible and orderable again		
TC-ADM-08	BR-03, SEC-002	Integration / Security	Customer tries to disable a book or adjust stock	Customer token	PATCH active and stock endpoints	active=false; delta=+100	403 for both; data unchanged		

Reports (UC-07)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-RPT-01	FR-013	Integration / Functional	Sales summary for a day	5 orders placed on 2026-10-01 totalling 3,450.00	GET /api/admin/reports/sales	from=to=2026-10-01	orderCount=5; revenue=3450.00		
TC-RPT-02	FR-013	Integration / Boundary	Date range is inclusive	Orders on 30 Sep, 1 Oct and 2 Oct	GET sales report for 1 Oct to 2 Oct	from=2026-10-01; to=2026-10-02	Only 1 Oct and 2 Oct orders counted		

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-RPT-03	FR-013, FR-012	Integration / Functional	Failed payments excluded	1 successful order, 1 FAILED payment	GET sales report	today	Count 1; failed payment not in revenue		
TC-RPT-04	FR-013	Integration / Boundary	Period with no orders	No orders in range	GET sales report	from=to=2020-01-01	200 with orderCount 0 and revenue 0.00		
TC-RPT-05	FR-013, SEC-003	Integration / Negative	Invalid dates	Admin logged in	GET sales report with bad parameters	from=2026-10-05; to=2026-10-01; from=abc	400 with message; no query run		
TC-RPT-06	FR-013	Integration / Functional	Low-stock report with default threshold	Books with stock 0, 3, 5, 6	GET /api/admin/reports/low-stock	default threshold 5	Books with stock 0, 3, 5 listed; 6 not listed		
TC-RPT-07	FR-013	Integration / Functional	Low-stock custom threshold and order	Same data	GET low-stock with threshold=3	threshold=3	Books with 0 and 3 only, sorted by lowest stock first		
TC-RPT-08	FR-013, SEC-002	Integration / Security	Customer requests a report	Customer token	GET both report endpoints	customer JWT	403 for both		

Security

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-SEC-01	SEC-002, BR-03	Integration / Security	RBAC sweep over all admin routes	Route table available; customer and anonymous tokens	Call every /api/admin/* route with customer token and with no token	generated from route list	Customer: 403; anonymous: 401; no data changed (new admin routes covered automatically)		
TC-SEC-02	SEC-006	Integration / Security	Security events are logged	API running	Cause a failed login, a 403 and an invalid JWT	wrong password; customer on admin route; tampered token	Each event stored with timestamp, user id (if known), IP and route; no passwords or tokens in logs		
TC-SEC-03	SEC-003	Integration + UI / Security	Stored XSS attempt	Admin and customer logged in	Save name and book title containing a script tag; view pages	alert(1)	Stored as text and shown escaped; no script runs		
TC-SEC-04	SEC-005	Review + Integration / Security	No card data stored	One successful payment made	Inspect payments collection, logs and API responses	tok_success	No card number, CVV or expiry anywhere; only gateway_ref and status		
TC-SEC-05	SEC-003, SEC-004	Integration / Security	NoSQL operator injection in login	API running	POST /api/auth/login with operator objects	{"email":{"\$ne":""},"password":{"\$ne":""}}	400; no login; no crash		
TC-SEC-06	SEC-004	Security scan / Security	Transport security and headers	Staging deployed	Request over HTTP; run TLS and header check	http:// URL; openssl s_client; response headers	HTTP redirected to HTTPS; TLS 1.2 or higher only; helmet headers present		
TC-SEC-07	SEC-003	Static analysis / Security	Dependency and code scan	CI pipeline configured	Run npm audit and SonarQube security analysis	main branch build	No High/Critical vulnerabilities open; security hotspots reviewed		
TC-SEC-08	SEC-002, SEC-003	Penetration (planned) / Security	Baseline scan of authentication and admin paths	Staging deployed with seeded users	Run OWASP ZAP baseline scan and manual checks on login, cart, checkout and admin routes	staging URL; test accounts	No High findings; Medium findings logged as defects		

Non-functional

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
TC-PERF-01	NFR-001	Performance / Performance	Catalog search response time	Staging seeded with 500 books; warmed up	Artillery: 50 virtual users for 5 minutes, mixed searches	title/author/genre queries	95th percentile <= 2.0 s; error rate < 1%		
TC-AVL-01	NFR-002	Operational / Availability	Availability over 7 days	Staging deployed; uptime monitor configured	Ping /api/health every 5 minutes inside the operating window	08:00-22:00 IST	Availability >= 99% excluding announced maintenance		
TC-USE-01	NFR-003	Walkthrough / Usability	New customer places an order	Staging deployed; 5 first-time users	Observe: find a book, add to cart, register, pay	each user's own choice of book	At least 4 of 5 succeed unaided, each within 5 primary screens		
TC-MNT-01	NFR-005	Review / Maintainability	Layering and unit-test check	Code on main branch	Code review checklist; SonarQube quality gate	services layer	No business logic in route files; each service has unit tests; line coverage on services >= 80%; branch coverage reported		
TC-SAF-01	SAF-003	Operational / Safety	Backup and restore	Daily backup job has run	Restore latest dump into an empty test database	last dump	Restore completes; users, books and orders counts match source		

System (E2E)

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
ST-01	FR-001, FR-002, FR-003	System / Functional	Visitor browses, registers and logs in	Staging seeded; no account	Open site; search a title; register; log in	new account details	Catalog usable before login; account created; logged-in state shown		
ST-02	FR-003, FR-005, FR-006, FR-007, FR-008, FR-009, FR-015, NFR-003	System / Functional	Complete purchase and tracking	Customer logged in	Search; open detail; add to cart; checkout; view confirmation; open order status	Book B1 qty 2; tok_success	Order confirmed with order number; total correct; status RECEIVED; five primary screens		
ST-03	FR-007, FR-012, BR-05	System / Negative	Payment failure then retry	Customer logged in; cart filled	Pay with a declined token; then pay with an approved token	tok_decline then tok_success	First attempt: failure message, cart kept, no order; second: order created		
ST-04	NFR-004, FR-007	System / Reliability	Payment timeout and retry	Mock gateway set to timeout once	Pay; wait for timeout; press Pay again	tok_timeout then tok_success	Exactly one order and one stock decrement		
ST-05	FR-011, BR-01, BR-03	System / Functional	Admin catalog lifecycle shown on storefront	Admin logged in	Add book; edit price; disable; enable; check storefront each time	Test Book 299.00	Storefront reflects each change within one page refresh		
ST-06	FR-009, FR-014, BR-02	System / Functional	Admin processes order, customer sees status	Order RECEIVED exists	Admin advances to DELIVERED; customer opens order after each step	3 status changes	Customer sees each status; invalid jump rejected		
ST-07	SEC-002, SEC-004, SEC-006	System / Security	RBAC sweep through the UI and API	Customer and anonymous sessions	Open admin pages and call admin APIs	customer JWT; no JWT	Redirect or 403/401 every time; events logged		

ID	Req(s)	Level / type	Description	Precondition	Steps	Test data	Expected result	Actual	Result
ST-08	FR-013, FR-010	System / Functional	Reports after several orders	Five orders placed by two customers	Open sales and low-stock reports; compare with history	seeded orders	Totals match order history; low-stock list matches stock levels		

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed; % passed / failed / blocked
- Defect density (defects per module) and defect aging (days open)
- Requirement coverage (requirements with at least one executed and passed test)
- Unit test line and branch coverage per module; SonarQube quality gate status
- Performance: p95 latency and error rate; availability percentage

Defect severity and handling:

Severity	Meaning	Example	Fix target
Critical	System unusable, data loss or security breach	Order created without payment; customer reaches admin API	Immediately; blocks release
High	Major requirement fails, no workaround	Stock goes negative; login fails	Within the sprint
Medium	Requirement partly fails, workaround exists	Wrong price-filter boundary	Before release
Low	Cosmetic or minor usability	Alignment, wording	If time permits

Defects are GitHub Issues (labels: bug, severity, failing TC ID). Workflow: New → Assigned → Fixed in a pull request referencing the issue → Verified by re-running the failing test and the regression suite → Closed. A fix adds or updates an automated test.

Reports: daily execution status during test weeks (a table from the CI results and execution record); final Test Summary Report with metrics, open defects, coverage, security and performance results, and RTM status.

15. Approvals

Role	Name	Signature / Date
QA Lead	S R Lokesh Adithya	
Dev Lead	Rishi Jain	
Product Owner		